

HRA-secure Attribute-based Proxy Re-encryption from Fully Homomorphic Attribute-based Encryption

Seiryu Hamasu¹, Hyungrok Jo², Shingo Sato^{1,2,3}, and Junji Shikata^{1,2}

¹ Graduate School of Environment and Information Sciences, Yokohama National University, Yokohama, Japan

² Institute of Advanced Sciences, Yokohama National University, Yokohama, Japan

³ Organization for the Promotion of Education, Yokohama National University, Yokohama, Japan

hamasu-seiryu-pw@ynu.jp

{jo-hyungrok-xz,sato-shingo-zk,shikata-junji-rb}@ynu.ac.jp

Abstract. Proxy re-encryption (PRE) allows a proxy to convert a ciphertext under one public key into a ciphertext under another public key. As an extension of PRE, attribute-based PRE (ABPRE) is an attribute-based variant of PRE that converts a ciphertext associated with an attribute vector (or a policy) into a ciphertext associated with a different attribute vector (or a different policy). Several lattice-based ABPRE schemes with security against honest re-encryption attacks (HRA security) have been proposed; however, all existing schemes have been found to have security flaws.

In this paper, we propose a provably HRA-secure ABPRE scheme. Our construction is generic and starts from bootstrappable fully homomorphic attribute-based encryption (FHABE) with circular security. By instantiating our generic construction with an existing lattice-based FHABE scheme, we obtain the first provably HRA-secure ABPRE from lattices.

Keywords: Proxy re-encryption · Attribute-based encryption · Fully homomorphic encryption

1 Introduction

1.1 Background and Related work

Cloud computing and large-scale distributed systems are now widely adopted for storing and processing sensitive data in domains such as healthcare, finance, and critical infrastructure. In these environments, *attribute-based encryption* (ABE) provides fine-grained access control by allowing data owners to encrypt data under expressive access policies defined over attributes, so that only users whose credentials satisfy the policy can decrypt. However, real-world deployments demand *dynamic* delegation of access rights: when organizational structures change, when data must be shared across institutional boundaries, or

when access policies need to be updated, all without re-encrypting the data from scratch and without exposing plaintext to the delegating server.

To enable dynamic delegation, Blaze, Bleumer, and Strauss [4] introduced the notion of *proxy re-encryption* (PRE). PRE allows a semi-trusted party (called a proxy) with a re-encryption key $rk_{A \rightarrow B}$ to convert ciphertexts under Alice’s public key pk_A into ciphertexts under Bob’s public key pk_B , without knowledge of any information about the underlying messages or the secret keys. Such re-encryption is useful in several applications including e-mail forwarding, encrypted data storage, and key escrow. PRE schemes are further classified by the following properties:

Unidirectional vs. Bidirectional. In *unidirectional* PRE, $rk_{A \rightarrow B}$ enables the conversion of ciphertexts under pk_A into ciphertexts under pk_B , but not vice versa. In contrast, *bidirectional* PRE allows re-encryption in both directions. We only consider unidirectional PRE in this paper, since it offers finer-grained control over re-encryption and greater flexibility in practice. Moreover, unidirectional PRE can be trivially converted into bidirectional PRE by providing the proxy with both $rk_{A \rightarrow B}$ and $rk_{B \rightarrow A}$.

Single-hop vs. Multi-hop. A PRE scheme is said to be *single-hop* if a re-encryption key allows the conversion only once, for an encryption of a message. Meanwhile, a PRE is said to be *multi-hop* if it allows the conversion many times, for a (re-)encryption of a message. Because multi-hop functionality offers greater flexibility in chained delegation, we focus on constructing a multi-hop PRE scheme.

HRA security. Cohen [9] demonstrated the inadequacy of CPA security for PRE: it guarantees security only against a proxy, and does not prevent Bob from learning information about Alice’s secret key from a ciphertext re-encrypted from Alice to Bob. To address this issue, Cohen [9] introduced security against honest re-encryption attacks (HRA), which is stronger than CPA security. The adversary in the HRA game is given access to a re-encryption oracle and an encryption oracle. This re-encryption oracle re-encrypts ciphertexts generated via the encryption oracle, and the adversary is allowed to obtain re-encryptions even from honest keys to corrupted keys. On the other hand, the adversary in the CPA game is not allowed to access such oracles and get any re-encryptions from honest keys to corrupt keys. We can observe that HRA security formalizes security against both a proxy and a delegatee (Bob), and thereby ensures a more realistic security for PRE applications.

Attribute-based proxy re-encryption. As an extension of PRE, Liang et al. [14] introduced the notion of *attribute-based proxy re-encryption* (ABPRE). ABPRE combines the static, fine-grained access control of ABE with the dynamic delegation capability of PRE, enabling a proxy to transform a ciphertext for one policy into a ciphertext for another policy. ABPRE is classified into key-policy ABPRE and ciphertext-policy ABPRE; throughout this paper, we focus on the key-policy setting, where users’ secret keys are embedded in policies. Furthermore, we consider post-quantum ABPRE.

Related work. Blaze, Bleumer and Strauss [4] introduced the notion of PRE and proposed a multi-hop bidirectional CPA-secure PRE scheme based on the

DDH assumption. Ateniese, Fu, Green and Hohenberger proposed a single-hop unidirectional CPA-secure PRE scheme with bilinear maps [3]. Ateniese, Benson and Hohenberger introduced the notion of key-privacy for PRE and presented a key-private CPA-secure PRE scheme based on pairings. There has also been extensive research on achieving stronger security notions for PRE. In particular, several PRE schemes with security against chosen ciphertext attacks (called CCA2 security) or its variant have been proposed so far (e.g., [6,15,8,5,11,16]) since Canetti and Hohenberger [6] formalized CCA2 security for PRE.

Cohen [9] introduced the notion of security against honest re-encryption attacks (HRA), by strengthening CPA security for PRE. This line of work was extended by Fuchsbauer et al. [12], who Fuchsbauer et al. [12] formalized CPA and HRA security under adaptive corruptions and constructed adaptively HRA-secure PRE schemes based on existing constructions such as [2,1,13,7]. Wan et al. [18] further improved this direction by proposing an adaptively HRA-secure PRE scheme with optimized properties. Zhou et al. [24] introduced fine-grained PRE, which integrates PRE with functional encryption, and Zhou, Liu, and Han [23] proposed an HRA-secure fine-grained PRE based on LWE.

As in (plain) PRE, several lattice-based ABPRE schemes with claimed HRA security have been proposed in [17,22,21,20]; however, Yin et al. [19] demonstrated attacks against HRA security of lattice-based PRE schemes whose re-encryption procedure is a linear transformation. By employing these attacks, adversaries in the HRA game can recover messages submitted in the challenge phase and break the HRA security of those lattice-based PRE including ABPRE schemes [17,22,21]. Meanwhile, Yin et al.’s attacks [19] do not work only for Zhao et al.’s ABPRE scheme [20].

1.2 Our Contribution

In this paper, we propose a HRA-secure ABPRE scheme. As one of our results, we identify a subtle gap in the security proof for the ABPRE scheme [20] constructed from fully homomorphic attribute-based encryption (FHABE). We further argue that this gap is not easily repairable; a detailed discussion is given in Section 3.3. Since attacks against the other ABPRE schemes [17,22,21] were shown by Yin et al. [19], our result shows that no existing ABPRE scheme has a valid proof of HRA security.

We then propose a generic construction of ABPRE from FHABE and show its HRA security. By instantiating our generic construction with Zhao et al.’s FHABE scheme [20], we obtain a concrete lattice-based HRA-secure ABPRE scheme. In summary, the proposed scheme is the first ABPRE scheme with provable HRA security.

Technical Overview. Our generic construction of HRA-secure ABPRE builds on the following line of work in the PRE setting:

- Cohen [9] introduced the notion of re-encryption simulatability and showed that PRE satisfying this property achieves HRA security.

- Fuchsbauer et al. [12] proposed a generic construction of PRE with (a variant of) re-encryption simulatability, starting from fully homomorphic encryption (FHE) with *sanitizability* [10].
- Ducas and Stehlé [10] showed that sanitizable FHE can be constructed from any (lattice-based) FHE.

Combining these results, we can observe that it is possible to construct an HRA-secure PRE by applying Fuchsbauer et al.’s generic construction to any FHE. We lift this approach from the PRE setting to the ABPRE setting: our goal is to construct HRA-secure ABPRE starting from FHABE.

A key obstacle is that sanitizability has only been defined for (plain) FHE, and no corresponding notion exists for FHABE. To overcome this, we formalize a suitable notion of sanitizability for FHABE by extending the definition of Ducas and Stehlé [10]. We then show that the FHABE scheme [20] satisfies this formalized sanitizability. Based on this, we present a construction of HRA-secure ABPRE, which starts from FHABE satisfying sanitizability, CPA security, and master-key circular security. By instantiating our construction with the existing lattice-based FHABE scheme of Zhao et al. [20], we can obtain a concrete lattice-based ABPRE scheme with provable HRA security.

2 Preliminaries

Notation. For an integer $n \in \mathbb{N}$, let $[n] = \{1, \dots, n\}$. We write $x \leftarrow \mathcal{X}$ to denote that x is sampled according to distribution $\mathcal{X}$, and $s \stackrel{\$}{\leftarrow} \mathcal{S}$ to denote that s is sampled uniformly at random from a set $\mathcal{S}$. Column vectors are denoted by bold lowercase letters (e.g., $\mathbf{a}$), and matrices by bold uppercase letters (e.g., $\mathbf{A}$). For a vector $\mathbf{a}$, let $\|\mathbf{a}\|$ and $\|\mathbf{a}\|_\infty$ be the l_2 norm and the maximum norm, respectively. For a matrix $\mathbf{A}$, let $\|\mathbf{A}\|$ be the l_2 norm of the longest column of $\mathbf{A}$.

A negligible function is denoted by $\text{negl}(\cdot)$. For a function $f : \mathcal{S} \rightarrow \mathcal{S}$ and an element $a \in \mathcal{S}$, we write $f^k(a)$ for the k -fold composition of f applied to a .

Statistical Distance. For two distributions $\mathcal{X}, \mathcal{Y}$ over a finite set $\mathcal{S}$, the statistical distance $\Delta(\mathcal{X}, \mathcal{Y})$ between $\mathcal{X}$ and $\mathcal{Y}$ is defined as

$$\Delta(\mathcal{X}, \mathcal{Y}) = \frac{1}{2} \sum_{s \in \mathcal{S}} |\Pr[\mathcal{X} = s] - \Pr[\mathcal{Y} = s]|.$$

Lemma 1 ([10], Lemma 2.3). *Let $f : \mathcal{S} \rightarrow \mathcal{S}$ be a probabilistic function and $\delta \in [0, 1]$ such that $\Delta(f(a), f(b)) \leq \delta$ for all $a, b \in \mathcal{S}$. Then, for all $a, b \in \mathcal{S}$ and $k \geq 0$, it holds that $\Delta(f^k(a), f^k(b)) \leq \delta^k$.*

2.1 Fully Homomorphic Attribute-based Encryption

Definition 1 (FHABE [20]). *Let $\mathcal{M}$ be a plaintext space, $\mathcal{X}^l$ be an attribute space, $\mathcal{F} = \{f : \mathcal{X}^l \rightarrow \mathbb{Z}\}$ be a policy family, and $\mathcal{G} = \{g : \mathcal{M}^* \rightarrow \mathcal{M}\}$ be a circuit family. A (key-policy) FHABE scheme consists of the following polynomial-time algorithms:*

- **Setup**($1^\lambda, l$) : On input a security parameter λ and l , output a public parameter pp and a master secret key msk .
- **KeyGen**($\text{pp}, \text{msk}, f \in \mathcal{F}$) : On input the public parameter pp , the master secret key msk , and a policy function $f \in \mathcal{F}$, output a public key pk_f and a secret key sk_f for policy f . Typically, pk_f is used to evaluate ciphertexts for policy f .
- **Enc**($\text{pp}, \mathbf{x} \in \mathcal{X}^l, \mu \in \mathcal{M}$) : On input the public parameter pp , an attribute $\mathbf{x} \in \mathcal{X}^l$, and a message $\mu \in \mathcal{M}$, output a ciphertext $\text{ct}_{\mathbf{x}}$ associated with $\mathbf{x}$.
- **Dec**($\text{pp}, \text{sk}_f, \text{ct}$) : On input the public parameter pp , a secret key sk_f for policy f , and a ciphertext ct , output an error symbol $\perp$ if ct does not satisfy policy f . Otherwise, output a message μ .
- **Eval**($\text{pp}, \text{pk}_f, (\text{ct}_1, \dots, \text{ct}_k), g$) : On input the public parameter pp , a public key pk_f for policy f , ciphertexts $\text{ct}_1, \dots, \text{ct}_k$ of messages $\mu_1, \dots, \mu_k$, respectively, and a circuit $g \in \mathcal{G}$, output $\perp$ if there exists $i \in [k]$ such that ct_i does not satisfy policy f . Otherwise, output an evaluated ciphertext $\hat{\text{ct}}_f$ of a message $g(\mu_1, \dots, \mu_k)$ for policy f .

Definition 2 (FHABE Correctness). A FHABE scheme $\text{FHABE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ is said to be correct if, for every $\lambda \in \mathbb{N}$, $l \in \mathbb{N}$, $f \in \mathcal{F}$, $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, l)$, and $(\text{pk}_f, \text{sk}_f) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f)$, the following conditions hold:

- For every message $\mu \in \mathcal{M}$, every attribute $\mathbf{x} \in \mathcal{X}^l$ such that $f(\mathbf{x}) = 0$, and every ciphertext $\text{ct}_{\mathbf{x}} \leftarrow \text{Enc}(\text{pp}, \mathbf{x}, \mu)$, it holds that $\Pr[\text{Dec}(\text{pp}, \text{sk}_f, \text{ct}_{\mathbf{x}}) = \mu] \geq 1 - \text{negl}(\lambda)$.
- For every k messages $\mu_1, \dots, \mu_k \in \mathcal{M}$, ciphertexts $\text{ct}_1, \dots, \text{ct}_k$ of messages $\mu_1, \dots, \mu_k$ for policy f , respectively, and circuit $g \in \mathcal{G}$, it holds that $\Pr[\text{Dec}(\text{pp}, \text{sk}_f, \hat{\text{ct}}_f) = g(\mu_1, \dots, \mu_k)] \geq 1 - \text{negl}(\lambda)$, where $\hat{\text{ct}}_f \leftarrow \text{Eval}(\text{pp}, \text{pk}_f, (\text{ct}_1, \dots, \text{ct}_k), g)$.

CPA security of FHABE is defined by the following game $\text{Exp}_{\text{FHABE}, \mathcal{A}}^{\text{CPA}}(\lambda)$ between an adversary $\mathcal{A}$ and the challenger:

1. **Init:** $\mathcal{A}$ sends the target attribute $\mathbf{x}^* \in \mathcal{X}^l$.
2. **Setup:** The challenger runs $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, l)$ and sends pp to $\mathcal{A}$.
3. **Query Phase 1:** $\mathcal{A}$ is allowed to make polynomially many queries to the key generation oracle $\mathcal{O}^{\text{KeyGen}}$ that takes a policy $f \in \mathcal{F}$ as input, runs $(\text{pk}_f, \text{sk}_f) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f)$, and outputs pk_f if $f(\mathbf{x}^*) = 0$ and $(\text{pk}_f, \text{sk}_f)$ otherwise.
4. **Challenge:** $\mathcal{A}$ submits two messages $\mu_0, \mu_1 \in \mathcal{M}$ with equal length. The challenger chooses a random bit $\beta \xleftarrow{\$} \{0, 1\}$ and outputs a challenge ciphertext $\text{ct}^* \leftarrow \text{Enc}(\text{pp}, \mathbf{x}^*, \mu_\beta)$.
5. **Query Phase 2:** After receiving the challenge ciphertext ct^* , $\mathcal{A}$ continues to make queries as in **Query Phase 1**.
6. **Guess:** $\mathcal{A}$ outputs $\beta' \in \{0, 1\}$. The game outputs 1 if $\beta = \beta'$ and 0 otherwise.

Definition 3 (CPA Security of FHABE). A FHABE scheme $\text{FHABE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ is selectively CPA-secure if, for any PPT adversary $\mathcal{A}$ in $\text{Exp}_{\text{FHABE}, \mathcal{A}}^{\text{CPA}}(\lambda)$, it holds that $\text{Adv}_{\text{FHABE}, \mathcal{A}}^{\text{CPA}}(\lambda) := |\Pr[\beta = \beta'] - 1/2| \leq \text{negl}(\lambda)$.

Furthermore, we define *user-key circular security* for FHABE. This security notion ensures that no adversary against a FHABE scheme can distinguish between an encryption of a user secret key and that of a random value.

The security game $\text{Exp}_{\text{FHABE}, \mathcal{A}}^{\text{uCircular}}(\lambda)$ is defined as follows:

1. **Init:** $\mathcal{A}$ sends the target attribute $\mathbf{x}^* \in \mathcal{X}^l$.
2. **Setup:** The challenger runs $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, l)$, sends pp to $\mathcal{A}$, and chooses $\beta \xleftarrow{\$} \{0, 1\}$ at random.
3. **Query Phase:** $\mathcal{A}$ is allowed to make polynomially many queries in any order to the following oracles:
 - $\mathcal{O}^{\text{KeyGen}}(f)$: On input a policy $f \in \mathcal{F}$, run $(\text{pk}_f, \text{sk}_f) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f)$ and output pk_f if $f(\mathbf{x}^*) = 0$. Otherwise, output $(\text{pk}_f, \text{sk}_f)$.
 - $\mathcal{O}^{\text{Enc}}(f)$: On input a policy $f \in \mathcal{F}$, output $\perp$ if $f(\mathbf{x}^*) \neq 0$. Otherwise, run $(\text{pk}_f, \text{sk}_f) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f)$ and do the following:
 - If $\beta = 0$, sample $\mu \xleftarrow{\$} \mathcal{M}$ at random such that $|\mu| = |\text{sk}_f|$, and compute $\text{ct}_{\mathbf{x}^*} \leftarrow \text{Enc}(\text{pp}, \mathbf{x}^*, \mu)$.
 - If $\beta = 1$, compute $\text{ct}_{\mathbf{x}^*} \leftarrow \text{Enc}(\text{pp}, \mathbf{x}^*, \text{sk}_f)$.
4. **Guess:** $\mathcal{A}$ outputs $\beta' \in \{0, 1\}$. The game outputs 1 if $\beta = \beta'$ and outputs 0 otherwise.

Definition 4 (User-key Circular Security). A FHABE scheme $\text{FHABE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ is user-key circular-secure if, for any PPT adversary $\mathcal{A}$ in $\text{Exp}_{\text{FHABE}, \mathcal{A}}^{\text{uCircular}}(\lambda)$, it holds that $\text{Adv}_{\text{FHABE}, \mathcal{A}}^{\text{uCircular}}(\lambda) := |\Pr[\beta = \beta'] - 1/2| \leq \text{negl}(\lambda)$.

We assume that bootstrappable FHABE schemes achieve this security notion, since bootstrapping inherently requires some auxiliary information related to the secret key. Indeed, in the FHABE scheme [20], the public key pk_f includes an encryption of the secret key sk_f , and its CPA-security is proven under the circular security assumption.

2.2 Attribute-based Proxy Re-encryption

Definition 5 (ABPRE). Let $\mathcal{M}$ be a plaintext space, $\mathcal{X}^l$ be an attribute space, and $\mathcal{F} = \{f : \mathcal{X}^l \rightarrow \mathbb{Z}\}$ be a policy family. A multi-hop unidirectional key-policy ABPRE scheme consists of Setup , KeyGen , Enc and Dec described in Definition 1, as well as the following additional polynomial-time algorithms:

- $\text{ReKeyGen}(\text{pp}, \text{sk}_{f_1}, \text{pk}_{f_2}, f_1, f_2)$: On input the public parameter pp , a secret key sk_{f_1} , a public key pk_{f_2} and a pair of policies $(f_1, f_2) \in \mathcal{F} \times \mathcal{F}$, output the re-encryption key $\text{rk}_{f_1 \rightarrow f_2}$. This key can re-encrypt a ciphertext for policy f_1 to a ciphertext for policy f_2 .

- $\text{ReEnc}(\text{pp}, \text{rk}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct})$: On input the public parameter pp , a re-encryption key $\text{rk}_{f_1 \rightarrow f_2}$, a public key pk_{f_2} , and a ciphertext ct , output $\perp$ if ct does not satisfy policy f_1 . Otherwise, output a re-encrypted ciphertext $\hat{\text{ct}}_{f_2}$ for policy f_2 .

Definition 6 (ABPRE Correctness). An ABPRE scheme $\text{ABPRE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{ReKeyGen}, \text{ReEnc})$ is said to be correct if, for every $\lambda \in \mathbb{N}$, $l \in \mathbb{N}$, and $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, l)$, the following conditions hold:

- For every $f \in \mathcal{F}$, $(\text{pk}_f, \text{sk}_f) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f)$, every message $\mu \in \mathcal{M}$, every attribute $\mathbf{x} \in \mathcal{X}^l$ such that $f(\mathbf{x}) = 0$, and every $\text{ct}_{\mathbf{x}} \leftarrow \text{Enc}(\text{pp}, \mathbf{x}, \mu)$, it holds that $\Pr[\text{Dec}(\text{pp}, \text{sk}_f, \text{ct}_{\mathbf{x}}) = \mu] \geq 1 - \text{negl}(\lambda)$.
- For every $(f_1, f_2) \in \mathcal{F} \times \mathcal{F}$, every $(\text{pk}_{f_1}, \text{sk}_{f_1}) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f_1)$, $(\text{pk}_{f_2}, \text{sk}_{f_2}) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f_2)$, $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \text{ReKeyGen}(\text{pp}, \text{sk}_{f_1}, \text{pk}_{f_2}, f_1, f_2)$, and every $\mu \in \mathcal{M}$, it holds that $\Pr[\text{Dec}(\text{pp}, \text{sk}_{f_2}, \text{ReEnc}(\text{pp}, \text{rk}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct})) = \mu] \geq 1 - \text{negl}(\lambda)$, where ct is a fresh or re-encrypted ciphertext of a message μ for policy f_1 .

HRA security of ABPRE is defined by the following game $\text{Exp}_{\text{ABPRE}, \mathcal{A}}^{\text{HRA}}(\lambda)$ between an adversary $\mathcal{A}$ and the challenger:

1. **Init:** $\mathcal{A}$ sends the target attribute $\mathbf{x}^* \in \mathcal{X}^l$.
2. **Setup:** The challenger runs $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, l)$ and sends pp to $\mathcal{A}$. Then the challenger initializes a counter $N_{ct} = 0$, empty key-value stores $\mathcal{CT}$, $\mathcal{RK}$, and an empty set $\mathcal{TG}$.
3. **Query Phase 1:** $\mathcal{A}$ is allowed to make polynomially many queries in any order to the following oracles:
 - $\mathcal{O}^{\text{KeyGen}}(f)$: On input a policy $f \in \mathcal{F}$, run $(\text{pk}_f, \text{sk}_f) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f)$ and output pk_f if $f(\mathbf{x}^*) = 0$ and $(\text{pk}_f, \text{sk}_f)$ otherwise.
 - $\mathcal{O}^{\text{ReKeyGen}}(f_1, f_2)$: On input a pair of policies $(f_1, f_2) \in \mathcal{F} \times \mathcal{F}$, output $\perp$ if $f_1(\mathbf{x}^*) = 0$ and $f_2(\mathbf{x}^*) \neq 0$. Otherwise, proceed as follows:
 - If $\mathcal{RK}[f_1, f_2]$ does not exist, run $(\text{pk}_{f_1}, \text{sk}_{f_1}) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f_1)$, $(\text{pk}_{f_2}, \text{sk}_{f_2}) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f_2)$, $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \text{ReKeyGen}(\text{pp}, \text{sk}_{f_1}, \text{pk}_{f_2}, f_1, f_2)$, and output $\text{rk}_{f_1 \rightarrow f_2}$. Then, set $\mathcal{RK}[f_1, f_2] \leftarrow \text{rk}_{f_1 \rightarrow f_2}$.
 - Otherwise, retrieve $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \mathcal{RK}[f_1, f_2]$ and output $\text{rk}_{f_1 \rightarrow f_2}$.
 - $\mathcal{O}^{\text{Enc}}(\mathbf{x}, \mu)$: On input an attribute $\mathbf{x} \in \mathcal{X}^l$ and a message μ , output $\text{ct}_{\mathbf{x}} \leftarrow \text{Enc}(\text{pp}, \mathbf{x}, \mu)$. Increments N_{ct} and sets $\mathcal{CT}[\mathbf{x}, N_{ct}] \leftarrow \text{ct}_{\mathbf{x}}$.
 - $\mathcal{O}^{\text{ReEnc}}(f_1, f_2, \mathbf{x}, k)$: On input a pair of policies $(f_1, f_2) \in \mathcal{F} \times \mathcal{F}$, and a key $(\mathbf{x} \in \mathcal{X}^l, k \leq N_{ct})$, output $\perp$ if $\mathcal{CT}[\mathbf{x}, k]$ does not exist. If $f_1(\mathbf{x}) \neq 0$, also output $\perp$. Otherwise, proceed as follows:
 - If $\mathcal{RK}[f_1, f_2]$ does not exist, generate $(\text{pk}_{f_1}, \text{sk}_{f_1}) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f_1)$, $(\text{pk}_{f_2}, \text{sk}_{f_2}) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f_2)$, and $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \text{ReKeyGen}(\text{pp}, \text{sk}_{f_1}, \text{pk}_{f_2}, f_1, f_2)$ and set $\mathcal{RK}[f_1, f_2] \leftarrow \text{rk}_{f_1 \rightarrow f_2}$. Otherwise, retrieve $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \mathcal{RK}[f_1, f_2]$.
 - Run $(\text{pk}_{f_2}, \text{sk}_{f_2}) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f_2)$, retrieve $\text{ct} \leftarrow \mathcal{CT}[\mathbf{x}, k]$, and compute $\hat{\text{ct}}_{f_2} \leftarrow \text{ReEnc}(\text{pp}, \text{rk}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct})$.
 - Output $\hat{\text{ct}}_{f_2}$, increment N_{ct} , and set $\mathcal{CT}[\mathbf{x}, N_{ct}] \leftarrow \hat{\text{ct}}_{f_2}$.

4. **Challenge:** $\mathcal{A}$ submits two messages $\mu_0, \mu_1 \in \mathcal{M}$ with equal length. The challenger chooses a random bit $\beta \xleftarrow{\$} \{0, 1\}$ and outputs a challenge ciphertext $\text{ct}^* \leftarrow \text{Enc}(\text{pp}, \mathbf{x}^*, \mu_\beta)$. The challenger also increments N_{ct} , adds N_{ct} to the set $\mathcal{TG}$, and sets $\mathcal{CT}[\mathbf{x}^*, N_{ct}] \leftarrow \text{ct}^*$.
5. **Query Phase 2:** After receiving the challenge ciphertext ct^* , $\mathcal{A}$ continues to make queries in the same way as **Query Phase 1** except for the following procedure of $\mathcal{O}^{\text{ReEnc}}$:
 - If $k \in \mathcal{TG}$ and $f_1(\mathbf{x}^*) = 0$ and $f_2(\mathbf{x}^*) \neq 0$, output $\perp$.
 - If $k \in \mathcal{TG}$ and the output of $\mathcal{O}^{\text{ReEnc}}$ is not $\perp$, add N_{ct} to the set $\mathcal{TG}$ after increment.
6. **Guess:** $\mathcal{A}$ outputs $\beta' \in \{0, 1\}$. The game outputs 1 if $\beta = \beta'$ and 0 otherwise.

Definition 7 (HRA Security of ABPRE). An ABPRE scheme $\text{ABPRE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{ReKeyGen}, \text{ReEnc})$ is said to be *selectively HRA-secure* if, for any PPT adversary $\mathcal{A}$ in $\text{Exp}_{\text{ABPRE}, \mathcal{A}}^{\text{HRA}}(\lambda)$, it holds that $\text{Adv}_{\text{ABPRE}, \mathcal{A}}^{\text{HRA}}(\lambda) := |\Pr[\beta = \beta'] - 1/2| \leq \text{negl}(\lambda)$.

Remark 1. All of the ABPRE security games defined in previous works do not specify whether the re-encryption keys are *persistent* or *ephemeral*, i.e., whether $\text{rk}_{f_1 \rightarrow f_2}$ is reused or freshly generated each time the same pair (f_1, f_2) is queried. This difference is crucial because both Yin et al.’s attack [19] and our cryptanalysis of Zhao et al.’s ABPRE scheme [20] are assumed the persistent re-encryption key model.

In our ABPRE security games, we introduce a key-value store $\mathcal{RK}$ in order to ensure that the re-encryption keys are persistent. It is worth noting that the persistent model better captures real-world scenarios, where a proxy uses the same $\text{rk}_{f_1 \rightarrow f_2}$ for multiple re-encryption requests from policy f_1 to policy f_2 .

Similarly, CPA security of ABPRE is defined by the game $\text{Exp}_{\text{ABPRE}, \mathcal{A}}^{\text{CPA}}(\lambda)$, which is identical to $\text{Exp}_{\text{ABPRE}, \mathcal{A}}^{\text{HRA}}(\lambda)$ except for the following procedures:

- The game does not use N_{ct} , $\mathcal{CT}$, and $\mathcal{TG}$.
- $\mathcal{A}$ is not given access to $\mathcal{O}^{\text{Enc}}$ and $\mathcal{O}^{\text{ReEnc}}$ in **Query Phases 1 and 2**.

Definition 8 (CPA Security of ABPRE). An ABPRE scheme $\text{ABPRE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{ReKeyGen}, \text{ReEnc})$ is said to be *selectively CPA-secure* if, for any PPT adversary $\mathcal{A}$ in $\text{Exp}_{\text{ABPRE}, \mathcal{A}}^{\text{CPA}}(\lambda)$, it holds that $\text{Adv}_{\text{ABPRE}, \mathcal{A}}^{\text{CPA}}(\lambda) := |\Pr[\beta = \beta'] - 1/2| \leq \text{negl}(\lambda)$.

Moreover, we describe the property of *re-encryption simulatability*. Informally, re-encryption simulatable ABPRE requires that a re-encrypted ciphertext $\hat{\text{ct}}_{f_2} \leftarrow \text{ReEnc}(\text{pp}, \text{rk}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct})$ can be simulated without knowledge of the secret key sk_{f_1} (and the re-encryption key $\text{rk}_{f_1 \rightarrow f_2}$ generated from sk_{f_1}). Susilo et al. [17] showed that if a CPA-secure ABPRE scheme satisfies re-encryption simulatability, it is also HRA-secure.

Definition 9 (Re-encryption Simulatability). An ABPRE scheme (Setup, KeyGen, Enc, Dec, ReKeyGen, ReEnc) is re-encryption simulatable if there exists a PPT algorithm ReEncSim such that

$$\Delta((\text{ReEncSim}(\text{aux}), \text{aux}), (\text{ReEnc}(\text{pp}, \text{rk}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct}), \text{aux})) \leq \text{negl}(\lambda),$$

for all $\mu \in \mathcal{M}$ with overwhelming probability over $\text{aux} = (\text{pp}, f_1, f_2, \text{pk}_{f_1}, \text{pk}_{f_2}, \text{sk}_{f_2}, \text{ct}, \mu)$, where $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, l)$, $(\text{pk}_{f_1}, \text{sk}_{f_1}) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f_1 \in \mathcal{F})$, $(\text{pk}_{f_2}, \text{sk}_{f_2}) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f_2 \in \mathcal{F})$, $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \text{ReKeyGen}(\text{pp}, \text{sk}_{f_1}, \text{pk}_{f_2}, f_1, f_2)$, and ct is an encryption of a message μ for policy f_1 .

Lemma 2 ([17], Theorem 2). An ABPRE scheme is selective HRA-secure if it is selective CPA-secure and re-encryption simulatable.

3 Cryptanalysis of Zhao et al.'s ABPRE scheme

In this section, we briefly review the construction of Zhao et al.'s ABPRE scheme [20] and show a subtle gap in the security proof for this ABPRE scheme.

3.1 Overview of Zhao et al.'s ABPRE

We review the construction of Zhao et al.'s ABPRE $\Pi_{\text{ZCA}^+}^{\text{ABPRE}} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{ReKeyGen}, \text{ReEnc})$. The basic FHABE scheme is $\Pi^{\text{FHABE}} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ which appears in Appendix A. Following [20], we describe the algorithms ReKeyGen and ReEnc. The system parameters are set as follows: For a security parameter λ and a family of policy circuits $\mathcal{F} = \{f : \mathbb{Z}_q^l \rightarrow \mathbb{Z}_q\}$ with depth bound $d_{\mathcal{F}}$, let $n = \text{poly}(\lambda)$, $q = 2^d$, $m = \Theta(n \log q)$, $\alpha_{\mathcal{F}} = (m+1)^{d_{\mathcal{F}}}$. Additionally, let $\mathbf{G} = \mathbf{I}_n \otimes (1, 2, \dots, 2^{\lfloor \log q \rfloor}) \in \mathbb{Z}_q^{n \times m}$ be a gadget matrix and χ be a B_χ -bounded error distribution. Set $\sigma = \omega(\alpha_{\mathcal{F}} \cdot \sqrt{\log m})$, where $\alpha_{\mathcal{F}} > \sqrt{n \log m}$ for ensuring the correctness of the trapdoor algorithms. Let $Q = B_g^{d_g}$ denote the modulus of Ring-LWE, for some base B_g and $N = q/2$ denote its dimension. A bootstrapping base B_r and a key switching base B_{ks} are chosen suitably for the correctness of $\Pi_{\text{ZCA}^+}^{\text{ABPRE}}$. Then, the algorithms ReKeyGen and ReEnc are described as follows:

- ReKeyGen($\text{pp}, \text{sk}_{f_1}, \text{pk}_{f_2}, f_1, f_2$) $\rightarrow \text{rk}_{f_1 \rightarrow f_2}$: Parse $\text{sk}_{f_1} = \mathbf{R}_{f_1} \in \mathbb{Z}_q^{2m \times m}$.
 1. Choose an attribute vector $\mathbf{y} = (y_1, \dots, y_l) \in \mathcal{X}^l$ such that $f_2(\mathbf{y}) = 0$.
 2. Choose $\mathbf{E}_1^{(r)} \xleftarrow{\$} \mathbb{Z}_q^{n \times 2md}$, $\bar{\mathbf{E}}_2^{(r)} \leftarrow \chi^{(l+1)m \times 2md}$, and $\mathbf{E}_3^{(r)} \leftarrow \chi^{m \times 2md}$.
 3. Compute $\bar{\mathbf{M}}_1^{(r)} \leftarrow [\mathbf{A} | \mathbf{B}_1 + y_1 \mathbf{G} | \dots | \mathbf{B}_l + y_l \mathbf{G}]^\top \mathbf{E}_1^{(r)} + \bar{\mathbf{E}}_2^{(r)}$ and $\mathbf{M}_2 \leftarrow \mathbf{U}^\top \mathbf{E}_1^{(r)} + \mathbf{E}_3^{(r)} - \text{P2}(\mathbf{R}_{f_1})^\top$.
 4. Compute $\mathbf{D}_{f_2, \mathbf{y}} \leftarrow \text{KeyEval}_{\mathbf{D}}(f_2, \mathbf{y})$, and let

$$\mathbf{M}_1^{(r)} = \begin{bmatrix} \mathbf{I}_{m \times m} & \mathbf{0}_{m \times m} \\ \mathbf{0}_{lm \times m} & \mathbf{D}_{f_2, \mathbf{y}} \end{bmatrix}^\top \cdot \bar{\mathbf{M}}_1^{(r)} = [\mathbf{A} | \mathbf{B}_{f_2}]^\top \mathbf{E}_1^{(r)} + \mathbf{E}_2^{(r)},$$

where $\mathbf{E}_2^{(r)} \in \mathbb{Z}_q^{2m \times 2md}$ is an error-matrix whose maximum norm is small, and $\text{KeyEval}_D(f_2, \mathbf{y})$ outputs $\mathbf{D}_{f_2, \mathbf{y}}$ whose maximum norm is small such that $[\mathbf{B}_1 + x_1 \mathbf{G}] \cdots [\mathbf{B}_l + x_l \mathbf{G}] \cdot \mathbf{D}_{f_2, \mathbf{y}} = \mathbf{B}_{f_2} + f_2(\mathbf{y}) \mathbf{G}$.

5. Output $\text{rk}_{f_1 \rightarrow f_2} = \begin{bmatrix} \mathbf{M}_1^{(r)} & \mathbf{0}_{2m \times m} \\ \mathbf{M}_2^{(r)} & \mathbf{I}_{m \times m} \end{bmatrix}$.
- $\text{ReEnc}(\text{pp}, \text{rk}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct}) \rightarrow \text{ct}_{f_2}^{(r)}$:
 1. If ct is a fresh ciphertext under $\mathbf{x}$, as $\text{ct}_{\mathbf{x}} = (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l, \mathbf{c}_{out})$, output $\perp$ if $f_1(\mathbf{x}) \neq 0$; otherwise, compute $\text{ct}_{f_1} \leftarrow \text{KeyEval}_{\text{ct}}(f_1, \mathbf{x}, \{\mathbf{B}_i, \mathbf{c}_i\}_{i \in [l]})$ and set $\hat{\text{ct}}_{f_1} = (\hat{\mathbf{c}}_{f_1}, \mathbf{c}_{out}) \in \mathbb{Z}_q^{3m}$, where $\hat{\mathbf{c}}_{f_1} = [\mathbf{c}_{in} \parallel \mathbf{c}_{f_1}]$.
 2. If ct is an evaluated or re-encrypted ciphertext for f , output $\perp$ if $f \neq f_1$; otherwise, let ct be $\hat{\text{ct}}_f = (\hat{\mathbf{c}}_{f_1} = [\mathbf{c}_{in} \parallel \mathbf{c}_{f_1}], \mathbf{c}_{out})$.
 3. Compute $\bar{\text{ct}}_{f_2}^{(r)} = \text{rk}_{f_1 \rightarrow f_2} \cdot \begin{bmatrix} \mathbf{G}^{-1}(\hat{\mathbf{c}}_{f_1}) \\ \mathbf{c}_{out} \end{bmatrix}$, where $\mathbf{G}^{-1}(\hat{\mathbf{c}}_{f_1})$ is a vector such that $\mathbf{G} \cdot \mathbf{G}^{-1}(\hat{\mathbf{c}}_{f_1}) = \hat{\mathbf{c}}_{f_1}$.
 4. Output $\hat{\text{ct}}_{f_2}^{(r)} \leftarrow \text{Bootstrap}(\text{pp}, \text{pk}_{f_2}, \bar{\text{ct}}_{f_2}^{(r)})$, where Bootstrap is a deterministic bootstrapping algorithm used in $\Pi_{ZCA+}^{\text{ABPRE}}$.

3.2 Security proof for $\Pi_{ZCA+}^{\text{ABPRE}}$

In the security proof [20, Theorem 3] for $\Pi_{ZCA+}^{\text{ABPRE}}$, the reduction under the lattice assumption uses simulation algorithms ($\text{Setup}_{\text{sim}}, \text{ReKeyGen}_{\text{sim}}, \text{ReEnc}_{\text{sim-1}}, \text{ReEnc}_{\text{sim-2}}$). In particular, we consider the case where the adversary queries $\mathcal{O}^{\text{ReKeyGen}}(\tilde{f}_1, \tilde{f}_2)$ and $\mathcal{O}^{\text{ReEnc}}(f_1, f_2, \tilde{\mathbf{x}}, k)$ for $(\tilde{f}_1, \tilde{f}_2, \tilde{\mathbf{x}})$ such that $\tilde{f}_1(\mathbf{x}^*) = \tilde{f}_2(\mathbf{x}^*) \wedge \tilde{\mathbf{x}} \neq \mathbf{x}^*$. We show that the simulation algorithms do not work in this case.

Following [20], we describe the overview of $\text{ReKeyGen}_{\text{sim}}$ and $\text{ReEnc}_{\text{sim-2}}$, because these algorithms are used to simulate the environment of the case.

- $\text{ReKeyGen}_{\text{sim}}(\text{pp}, f_1, f_2)$ (where $f_1(\mathbf{x}^*) = 0$):
 1. Choose $\mathbf{y} = (y_1, \dots, y_l)$ such that $f_2(\mathbf{y}) = 0$.
 2. If $f_2(\mathbf{x}^*) \neq 0$, output $\perp$; otherwise, choose $\bar{\mathbf{X}}_1 \xleftarrow{\$} \mathbb{Z}_q^{(l+1)m \times 2md}$ and $\mathbf{X}_2^{(r)} \xleftarrow{\$} \mathbb{Z}_q^{m \times 2md}$.
 3. Evaluate $\mathbf{D}_{f_2, \mathbf{y}} \leftarrow \text{KeyEval}_D(f_2, \mathbf{y}, \{\mathbf{B}_1, \dots, \mathbf{B}_l\})$ and compute $\mathbf{X}_1^{(r)} = \begin{bmatrix} \mathbf{I}_{m \times m} & \mathbf{0}_{m \times m} \\ \mathbf{0}_{lm \times m} & \mathbf{D}_{f_2, \mathbf{y}} \end{bmatrix}^\top \cdot \bar{\mathbf{X}}_1^{(r)}$.
 4. Output $\text{rk}_{f_1 \rightarrow f_2}^{(\text{sim})} = \begin{bmatrix} \mathbf{X}_1^{(r)} & \mathbf{0}_{2m \times m} \\ \mathbf{X}_2^{(r)} & \mathbf{I}_{m \times m} \end{bmatrix}$.
- $\text{ReEnc}_{\text{sim-2}}(\text{pp}, f_1, f_2, (\mathbf{x}, j))$ (where $(\mathbf{x} \neq \mathbf{x}^* \wedge f_1(\mathbf{x}^*) = 0)$):
 1. Retrieve $\text{ct}_{\mathbf{x}} = (\mathbf{c}, \mathbf{c}_{out})$ corresponding to $(\mathbf{x}, j)$ from CT , where $\mathbf{c} = [\mathbf{c}_{in} \parallel \mathbf{c}_1 \parallel \dots \parallel \mathbf{c}_l]$.
 2. If $f_1(\mathbf{x}) \neq 0$, output $\perp$.
 3. If $f_1(\mathbf{x}) = 0$, since $\mathbf{x} \neq \mathbf{x}^*$, there must be some k such that $x_k \neq x_k^*$. Suppose that $x_l \neq x_l^*$ and proceed as follows:

- (a) Set $\mathbf{H}_x = [\mathbf{A}|\mathbf{B}_1+x_1\mathbf{G}|\cdots|\mathbf{B}_l+x_l\mathbf{G}] = [\mathbf{A}|\mathbf{A}\mathbf{S}_1^*+(x_1-x_1^*)\mathbf{G}|\cdots|\mathbf{A}\mathbf{S}_l^*+(x_l-x_l^*)\mathbf{G}]$.
- (b) Compute the trapdoor $\mathbf{T}_{[\mathbf{A}|\mathbf{B}_l+x_l\mathbf{G}]}$ by using the trapdoor algorithm, then obtain the trapdoor $\mathbf{T}_{[\mathbf{A}|\mathbf{B}_1+x_1\mathbf{G}|\cdots|\mathbf{B}_l+x_l\mathbf{G}]}$.
- (c) Generate the trapdoor $\mathbf{T}_{\mathbf{H}_x}$ by switching the rows of the matrix $\mathbf{T}_{[\mathbf{A}|\mathbf{B}_1+x_1\mathbf{G}|\cdots|\mathbf{B}_l+x_l\mathbf{G}]}$.
- (d) Sample $\mathbf{R}_x \leftarrow \text{SampleD}(\mathbf{H}_x, \mathbf{T}_{\mathbf{H}_x}, \mathbf{U}, \sigma)$ such that $\mathbf{H}_x \cdot \mathbf{R}_x = \mathbf{U}$.
- (e) Select an attribute vector $\mathbf{y} = (y_1, \dots, y_l)$ such that $f_2(\mathbf{y}) = 0$.
- (f) Set $\mathbf{H}_y = [\mathbf{A}|\mathbf{B}_1+y_1\mathbf{G}|\cdots|\mathbf{B}_l+y_l\mathbf{G}]$.
- (g) Choose a random matrix $\mathbf{R}_1 \xleftarrow{\$} \mathbb{Z}_q^{n \times (l+1)md}$, sample $\mathbf{R}_2 \leftarrow \chi^{(l+1)m \times (l+1)md}$ and $\mathbf{R}_3 \leftarrow \chi^{m \times (l+1)md}$.
- (h) Compute $\text{rk}_{x \rightarrow y}^{(\text{sim})} \leftarrow \begin{bmatrix} [\mathbf{A}|\mathbf{B}_1+y_1\mathbf{G}|\cdots|\mathbf{B}_l+y_l\mathbf{G}]^\top \mathbf{R}_1 + \mathbf{R}_2 & \mathbf{0}_{(l+1)m \times m} \\ \mathbf{U}^\top \mathbf{R}_1 + \mathbf{R}_3 - \text{P2}(\mathbf{R}_x)^\top & \mathbf{I}_{m \times m} \end{bmatrix}$.
- (i) Compute $\bar{\text{ct}}_y^{(r)} = \text{rk}_{x \rightarrow y}^{(r)} \cdot \begin{bmatrix} \mathbf{G}^{-1}(\mathbf{c}) \\ \mathbf{c}_{out} \end{bmatrix} \in \mathbb{Z}_q^{(l+2)m}$.
- (j) Output $\hat{\text{ct}}_{f_2}^{(r)} \leftarrow \text{Bootstrap}(\text{pp}, \text{pk}_{f_2}, \bar{\text{ct}}_y^{(r)})$.

3.3 Analysis of the security proof for $\Pi_{\text{ZCA}^+}^{\text{ABPRE}}$

We show that the simulation with the algorithms $\text{ReKeyGen}_{\text{sim}}$ and $\text{ReEnc}_{\text{sim-2}}$ does not work in the security proof (i.e., the proof of [20, Theorem 3]). As mentioned earlier, we consider policies and an attribute vector $(\tilde{f}_1, \tilde{f}_2, \tilde{\mathbf{x}})$ such that $\tilde{f}_1(\mathbf{x}^*) = \tilde{f}_2(\mathbf{x}^*) \wedge \tilde{\mathbf{x}} \neq \mathbf{x}^*$. Then, we demonstrate that a re-encrypted ciphertext generated by $\mathcal{O}^{\text{ReKeyGen}}(\tilde{f}_1, \tilde{f}_2)$ and $\mathcal{O}^{\text{ReEnc}}(\tilde{f}_1, \tilde{f}_2, (\tilde{\mathbf{x}}, \tilde{k}))$ is not correct.

1. Query $\mathcal{O}^{\text{ReKeyGen}}(\tilde{f}_1, \tilde{f}_2)$ and obtain $\text{rk}_{\tilde{f}_1 \rightarrow \tilde{f}_2}^{(\text{sim})}$.
2. Issue an encryption query $(\mathbf{x}, \mu)$ such that $\mu \neq \mu^* \wedge \tilde{f}_1(\mathbf{x}) = 0 \wedge \mathbf{x} \neq \mathbf{x}^*$ and obtain $(\tilde{k}, \tilde{\text{ct}}_x)$.
3. Issue a re-encryption query $(\tilde{f}_1, \tilde{f}_2, \tilde{k})$ and obtain $\tilde{\text{ct}}_{\tilde{f}_2}$.
4. Compute $\tilde{\text{ct}}'_{\tilde{f}_2} \leftarrow \text{ReEnc}(\text{pp}, \text{rk}_{\tilde{f}_1 \rightarrow \tilde{f}_2}^{(\text{sim})}, \text{pk}_{\tilde{f}_2}, \tilde{\text{ct}}_{x^*})$.

Then, the simulation with $\text{ReKeyGen}_{\text{sim}}(\text{pp}, f_1, f_2)$ and $\text{ReEnc}_{\text{sim-2}}(\text{pp}, f_1, f_2, (\mathbf{x}, j))$ fails in the HRA game, with overwhelming probability.

First, supposing that $\tilde{\text{ct}}_{\tilde{f}_2} \leftarrow \text{Bootstrap}(\text{pp}, \text{pk}_{f_2}, \bar{\text{ct}}_y^{(r)})$, $\bar{\text{ct}}_y^{(r)}$ is

$$\bar{\text{ct}}_y^{(r)} = \begin{bmatrix} [\mathbf{A}|\mathbf{B}_1+y_1\mathbf{G}|\cdots|\mathbf{B}_l+y_l\mathbf{G}]^\top \mathbf{R}_1 + \mathbf{R}_2 & \mathbf{0}_{(l+1)m \times m} \\ \mathbf{U}^\top \mathbf{R}_1 + \mathbf{R}_3 - \text{P2}(\mathbf{R}_x)^\top & \mathbf{I}_{m \times m} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{G}^{-1}(\mathbf{c}) \\ \mathbf{c}_{out} \end{bmatrix}.$$

Next, by letting $\tilde{\text{ct}}'_{\tilde{f}_2} \leftarrow \text{Bootstrap}(\text{pp}, \text{pk}_{f_2}, \bar{\text{ct}}_y^{(r)'})$, $\bar{\text{ct}}_y^{(r)'}$ is

$$\bar{\text{ct}}_y^{(r)'} = \text{rk}_{x^* \rightarrow y}^{(\text{sim})} \cdot \begin{bmatrix} \mathbf{G}^{-1}(\mathbf{c}) \\ \mathbf{c}_{out} \end{bmatrix} = \begin{bmatrix} \mathbf{X}_1^{(r)'} & \mathbf{0}_{(l+1)m \times m} \\ \mathbf{X}_2^{(r)'} & \mathbf{I}_{m \times m} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{G}^{-1}(\mathbf{c}) \\ \mathbf{c}_{out} \end{bmatrix}.$$

The algorithm Bootstrap actually performs the homomorphic AND operation on $\bar{\text{ct}}_y^{(r)'}$ (denoted as $\hat{\text{ct}}_{f_2}^{(r)} \leftarrow \text{Eval.AND}(\text{pp}, \text{pk}_{f_2}, \bar{\text{ct}}_y^{(r)'}, \bar{\text{ct}}_y^{(r)'})$). Additionally,

recall that the algorithm Eval.AND (i.e., Bootstrap) is deterministic. $\bar{\text{ct}}_{\mathbf{y}}^{(r)} \neq \bar{\text{ct}}_{\mathbf{y}}^{(r)'} holds with overwhelming probability. Hence, $\tilde{\text{ct}}_{\tilde{f}_2} \neq \tilde{\text{ct}}_{\tilde{f}_2}' holds with overwhelming probability, for $\tilde{\text{ct}}_{\tilde{f}_2} = \text{Eval.AND}(\text{pp}, \text{pk}_{\tilde{f}_2}, \bar{\text{ct}}_{\mathbf{y}}^{(r)}, \bar{\text{ct}}_{\mathbf{y}}^{(r)})$ and $\tilde{\text{ct}}_{\tilde{f}_2}' = \text{Eval.AND}(\text{pp}, \text{pk}_{\tilde{f}_2}, \bar{\text{ct}}_{\mathbf{y}}^{(r)}, \bar{\text{ct}}_{\mathbf{y}}^{(r)'})$.$$

Furthermore, it is difficult to fix the simulation by using $\text{rk}_{\tilde{f}_1 \rightarrow \tilde{f}_2}^{(\text{sim})}$ in the $\text{ReEnc}_{\text{sim-2}}$ simulation, in the straightforward way. Due to $\mathbf{x} \neq \mathbf{x}^*$, the adversary can re-encrypt the transformed ciphertext $\tilde{\text{ct}}_{\tilde{f}_2}$ into a ciphertext which can be decrypted by using a corrupted secret key sk_{f_3} such that $f_3(\mathbf{x}^*) \neq 0$. Additionally, the adversary can decrypt $\tilde{\text{ct}}_{\tilde{f}_2}$ and confirm if the message μ can be recovered correctly. Then, $\tilde{\text{ct}}_{\tilde{f}_2}$ must be a valid encryption of μ , but it is impossible to produce such an encryption by re-encryption via $\text{rk}_{\tilde{f}_1 \rightarrow \tilde{f}_2}^{(\text{sim})}$.

Therefore, the security proof for $\Pi_{\text{ZCA}^+}^{\text{ABPRE}}$ is not correct, and it is difficult to fix this proof in a straightforward way.

4 Sanitizability for FHABE

In this section, we introduce the notion of *sanitizability* for (F)HABE and show that the FHABE scheme [20] (with the modified parameters) satisfies this property. Sanitizable FHABE has an additional algorithm Sanitize , which transforms ciphertexts into the sanitized ciphertexts. For any two ciphertexts encrypting the same message under the same policy, the sanitized ciphertexts are required to be statistically indistinguishable.

Definition 10 (Sanitizability). *A FHABE scheme $(\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ is sanitizable if there exists a PPT algorithm Sanitize that, given a public parameter pp , a public key pk_f , and a ciphertext ct as input, outputs a sanitized ciphertext $\hat{\text{ct}}_f$. Here, Sanitize is required to satisfy the following properties for every $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, l)$, $f \in \mathcal{F}$, $(\text{pk}_f, \text{sk}_f) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f)$, and $\mu \in \mathcal{M}$:*

- Let ct be an encryption of message μ for policy f . Then, it holds that

$$\Pr[\text{Dec}(\text{pp}, \text{sk}_f, \text{Sanitize}(\text{pp}, \text{pk}_f, \text{ct})) = \mu] \geq 1 - \text{negl}(\lambda).$$

- Let ct_1, ct_2 be ciphertexts of message μ for policy f . Then, it holds that

$$\Delta((\hat{\text{ct}}_{1,f}, \text{pp}, \text{pk}_f, \text{sk}_f), (\hat{\text{ct}}_{2,f}, \text{pp}, \text{pk}_f, \text{sk}_f)) \leq 2^{-\lambda},$$

where $\hat{\text{ct}}_{1,f} \leftarrow \text{Sanitize}(\text{pp}, \text{pk}_f, \text{ct}_1)$ and $\hat{\text{ct}}_{2,f} \leftarrow \text{Sanitize}(\text{pp}, \text{pk}_f, \text{ct}_2)$.

4.1 Construction

We then present an algorithm for Sanitize by extending the algorithm Sanitize for FHE [10].

We define $\text{Sanitize}(\text{pp}, \text{pk}_f, \text{ct}) \rightarrow \hat{\text{ct}}_f$ as the λ -fold iteration a subprocedure $\text{Wash}(\text{pp}, \text{pk}_f, \text{ct}) \rightarrow \hat{\text{ct}}$. Here, Wash is required to satisfy the following properties:

- If ct is a valid ciphertext of a message μ for policy f , then $\hat{\text{ct}} \leftarrow \text{Wash}(\text{pp}, \text{pk}_f, \text{ct})$ is also a valid ciphertext of the message μ for policy f .
- For any valid ciphertexts ct_1, ct_2 of the same message μ for policy f , it holds that $\Delta(\text{Wash}(\text{pp}, \text{pk}_f, \text{ct}_1), \text{Wash}(\text{pp}, \text{pk}_f, \text{ct}_2)) \leq 1/2$.

Based on these properties, $\hat{\text{ct}}_f \leftarrow \text{Sanitize}(\text{pp}, \text{pk}_f, \text{ct})$ is a valid ciphertext, and we have

$$\Delta(\text{Sanitize}(\text{pp}, \text{pk}_f, \text{ct}_1), \text{Sanitize}(\text{pp}, \text{pk}_f, \text{ct}_2)) \leq 2^{-\lambda}.$$

by Lemma 1. Hence, this algorithm satisfies the properties in Definition 10.

4.2 Applying Sanitize to the FHABE scheme [20]

Now, we apply our construction of Sanitize to the FHABE scheme $\Pi_{\text{ZCA}+}^{\text{FHABE}}$ proposed by Zhao et al. [20] with modified parameters. Specifically, we define $\text{Wash}(\text{pp}, \text{pk}_f, \text{ct}) := \text{Rerand}(\text{pp}, \text{pk}_f, \text{Bootstrap}(\text{pp}, \text{pk}_f, \text{ct}))$, and describe how to construct Rerand that satisfies the conditions of Wash. The full construction of $\Pi_{\text{ZCA}+}^{\text{FHABE}}$ is given in Appendix A.

In $\Pi_{\text{ZCA}+}^{\text{FHABE}}$, for a ciphertext ct of a message $\mu \in \{0, 1\}^m$ for the policy f , $\bar{\text{ct}}_f \leftarrow \text{Bootstrap}(\text{pp}, \text{pk}_f, \text{ct})$ is of the form

$$\bar{\text{ct}}_f = \begin{bmatrix} \mathbf{c}_f \in \mathbb{Z}_q^{2m} \\ \mathbf{c}_{out} \in \mathbb{Z}_q^m \end{bmatrix} = \begin{bmatrix} [\mathbf{A}|\mathbf{B}_f]^\top \mathbf{s} + \mathbf{e} \\ \mathbf{U}^\top \mathbf{s} + \mathbf{e}_{out} + \lfloor q/4 \rfloor \cdot \mu \end{bmatrix} \in \mathbb{Z}_q^{3m},$$

where $\mathbf{A}, \mathbf{B}_f, \mathbf{U} \in \mathbb{Z}_q^{n \times m}$ are public matrices, $\mathbf{s} \in \mathbb{Z}_q^n$ is a secret vector, and $\mathbf{e} \in \mathbb{Z}_q^{2m}$, $\mathbf{e}_{out} \in \mathbb{Z}_q^m$ are error vectors such that $\|\mathbf{e}\| \leq 2m^2 B_\chi \cdot \log_2 q$ and $\|\mathbf{e}_{out}\| \leq 2m^2 \alpha_{\mathcal{F}} B_\chi \cdot \log_2 q + \delta$.

Given a bootstrapped ciphertext $\bar{\text{ct}}_f = (\mathbf{c}_f, \mathbf{c}_{out})$, we construct Rerand as follows:

1. Sample $\tilde{\mathbf{s}} \xleftarrow{\$} \mathbb{Z}_q^n$, $\tilde{\mathbf{e}} \xleftarrow{\$} [-E, E]^{2m}$, $\tilde{\mathbf{e}}_{out} \xleftarrow{\$} [-E, E]^m$ uniformly at random.
2. Compute $\tilde{\mathbf{c}}_f \leftarrow [\mathbf{A}|\mathbf{B}_f]^\top \tilde{\mathbf{s}} + \tilde{\mathbf{e}}$ and $\tilde{\mathbf{c}}_{out} \leftarrow \mathbf{U}^\top \tilde{\mathbf{s}} + \tilde{\mathbf{e}}_{out}$.
3. Output $\hat{\text{ct}}_f = \begin{bmatrix} \hat{\mathbf{c}}_f \\ \hat{\mathbf{c}}_{out} \end{bmatrix} \leftarrow \begin{bmatrix} \mathbf{c}_f + \tilde{\mathbf{c}}_f \\ \mathbf{c}_{out} + \tilde{\mathbf{c}}_{out} \end{bmatrix}$.

Next, we show that $\hat{\text{ct}}_f \leftarrow \text{Wash}(\text{pp}, \text{pk}_f, \text{ct})$ described above can be decrypted correctly. In $\Pi_{\text{ZCA}+}^{\text{FHABE}}$, the secret key sk_f is a matrix $\mathbf{R}_f \in \mathbb{Z}^{2m \times m}$ such that $[\mathbf{A}|\mathbf{B}_f]\mathbf{R}_f = \mathbf{U}$ and $\|\mathbf{R}_f\| \leq 2m\sigma$. The decryption algorithm $\text{Dec}(\text{pp}, \text{sk}_f, \hat{\text{ct}}_f)$ computes the following:

$$\begin{aligned} \hat{\mathbf{c}}_{out} - \mathbf{R}_f^\top \hat{\mathbf{c}}_f &= \mathbf{U}^\top (\mathbf{s} + \tilde{\mathbf{s}}) + \mathbf{e}_{out} + \tilde{\mathbf{e}}_{out} + \lfloor q/4 \rfloor \cdot \mu - \mathbf{R}_f^\top ([\mathbf{A}|\mathbf{B}_f]^\top (\mathbf{s} + \tilde{\mathbf{s}}) + \mathbf{e} + \tilde{\mathbf{e}}) \\ &= \mathbf{U}^\top (\mathbf{s} + \tilde{\mathbf{s}}) + \mathbf{e}_{out} + \tilde{\mathbf{e}}_{out} + \lfloor q/4 \rfloor \cdot \mu - \mathbf{U}^\top (\mathbf{s} + \tilde{\mathbf{s}}) - \mathbf{R}_f^\top (\mathbf{e} + \tilde{\mathbf{e}}) \\ &= \mathbf{e}_{out} + \tilde{\mathbf{e}}_{out} - \mathbf{R}_f^\top (\mathbf{e} + \tilde{\mathbf{e}}) + \lfloor q/4 \rfloor \cdot \mu. \end{aligned}$$

If the norm of the error term $\mathbf{e}_{out} + \tilde{\mathbf{e}}_{out} - \mathbf{R}_f^\top(\mathbf{e} + \tilde{\mathbf{e}})$ satisfies

$$\begin{aligned} & \|\mathbf{e}_{out} + \tilde{\mathbf{e}}_{out} - \mathbf{R}_f^\top(\mathbf{e} + \tilde{\mathbf{e}})\| \\ & \leq 2m^2\alpha_{\mathcal{F}}B_{\chi} \cdot \log_2 q + \delta + E + 2m\sigma(2m^2B_{\chi} \cdot \log_2 q + E) \\ & = 2m^2B_{\chi}(\alpha_{\mathcal{F}} + 2m\sigma) \cdot \log_2 q + \delta + (2m\sigma + 1)E < q/16, \end{aligned} \quad (1)$$

then $\hat{\mathbf{c}}_f$ (and its homomorphic evaluated ciphertext) can be decrypted by computing $\boldsymbol{\mu} \leftarrow \lfloor (4/q)(\mathbf{c}_{out} - \mathbf{R}_f^\top \hat{\mathbf{c}}_f) \rfloor \bmod 4$.

Finally, for encryptions $\mathbf{ct}_1, \mathbf{ct}_2$ of a message $\boldsymbol{\mu} \in \{0, 1\}^m$ for the policy f , we estimate the bound of $\Delta(\text{Wash}(\text{pp}, \text{pk}_f, \mathbf{ct}_1), \text{Wash}(\text{pp}, \text{pk}_f, \mathbf{ct}_2))$.

Let $\mathbf{s}_i \in \mathbb{Z}_q^n$, $\mathbf{e}_i \in \mathbb{Z}_q^{2m}$, $\mathbf{e}_{i,out} \in \mathbb{Z}_q^m$, $\tilde{\mathbf{s}}_i \xleftarrow{\$} \mathbb{Z}_q^n$, $\tilde{\mathbf{e}}_i \xleftarrow{\$} [-E, E]^{2m}$, and $\tilde{\mathbf{e}}_{i,out} \xleftarrow{\$} [-E, E]^m$ for $i \in \{1, 2\}$. Then, $\hat{\mathbf{c}}_{i,f} \leftarrow \text{Wash}(\text{pp}, \text{pk}_f, \mathbf{ct}_i)$ is of the form

$$\hat{\mathbf{c}}_{i,f} = \begin{bmatrix} \hat{\mathbf{c}}_{i,f} \\ \hat{\mathbf{c}}_{i,out} \end{bmatrix} = \begin{bmatrix} [\mathbf{A}|\mathbf{B}_f]^\top(\mathbf{s}_i + \tilde{\mathbf{s}}_i) + \mathbf{e}_i + \tilde{\mathbf{e}}_i \\ \mathbf{U}^\top(\mathbf{s}_i + \tilde{\mathbf{s}}_i) + \mathbf{e}_{i,out} + \tilde{\mathbf{e}}_{i,out} + \lfloor q/4 \rfloor \cdot \boldsymbol{\mu} \end{bmatrix}.$$

Since $(\mathbf{s}_i + \tilde{\mathbf{s}}_i)$ follows the uniform distribution over $\mathbb{Z}_q^n$, the statistical difference between $\hat{\mathbf{c}}_{1,f}$ and $\hat{\mathbf{c}}_{2,f}$ lies only in the error term $\mathbf{e}_i$, $\mathbf{e}_{i,out}$, $\tilde{\mathbf{e}}_i$, and $\tilde{\mathbf{e}}_{i,out}$. Hence, we have

$$\begin{aligned} \Delta(\hat{\mathbf{c}}_{1,f}, \hat{\mathbf{c}}_{2,f}) & \leq \Delta(\hat{\mathbf{c}}_{1,f}, \hat{\mathbf{c}}_{2,f}) + \Delta(\hat{\mathbf{c}}_{1,out}, \hat{\mathbf{c}}_{2,out}) \\ & \leq \Delta(\mathbf{e}_1 + \tilde{\mathbf{e}}_1, \mathbf{e}_2 + \tilde{\mathbf{e}}_2) + \Delta(\mathbf{e}_{1,out} + \tilde{\mathbf{e}}_{1,out}, \mathbf{e}_{2,out} + \tilde{\mathbf{e}}_{2,out}) \\ & \leq 2m \cdot \frac{2m^2B_{\chi} \cdot \log_2 q}{E} + \frac{2m^2\alpha_{\mathcal{F}}B_{\chi} \cdot \log_2 q + \delta}{E}. \end{aligned}$$

If $\Delta(\hat{\mathbf{c}}_{1,f}, \hat{\mathbf{c}}_{2,f}) \leq 1/2$ holds, then we have

$$E \geq 4m^3B_{\chi}(\alpha_{\mathcal{F}} + 2) + 2m\delta. \quad (2)$$

Therefore, the FHABE scheme $\Pi_{\text{ZCA}+}^{\text{FHABE}}$ with modified parameters that satisfy inequalities (1) and (2) is sanitizable. Note that parameters in the original scheme $\Pi_{\text{ZCA}+}^{\text{FHABE}}$ are required that $2m^2B_{\chi}(\alpha_{\mathcal{F}} + 2m\sigma) \cdot \log_2 q + \delta < q/16$.

The overhead of the proposed algorithm **Sanitize** is dominated by λ iterations of the bootstrapping procedure. In $\Pi_{\text{ZCA}+}^{\text{FHABE}}$, bootstrapping takes $O(n \log^2 q \log^2 Q)$ time [20], where n is the LWE dimension, and q and Q are the LWE and RLWE moduli, respectively. Therefore, the time complexity of **Sanitize** is $O(\lambda \cdot n \log^2 q \log^2 Q)$.

5 Our HRA-secure ABPRE

In this section, we propose our HRA-secure ABPRE scheme $\text{ABPRE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{ReKeyGen}, \text{ReEnc})$. This is a generic construction starting from a sanitizable FHABE scheme $\text{FHABE} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval}, \text{Sanitize})$. We assume that ABPRE and FHABE are defined over the same message space $\mathcal{M}$, the same attribute space $\mathcal{X}^l$, and the same policy family $\mathcal{F} = \{f : \mathcal{X}^l \rightarrow \mathbb{Z}\}$.

Since the algorithms Setup, KeyGen, Enc, Dec of ABPRE are identical to those of FHABE, we omit their descriptions. The additional algorithms ReKeyGen, ReEnc of ABPRE are constructed as follows:

- ReKeyGen($\text{pp}, \text{sk}_{f_1}, \text{pk}_{f_2}, f_1, f_2$) : On input the public parameter pp , a secret key sk_{f_1} , a public key $\text{pk}_{f_2} = \text{pk}'_{f_2}$ and a pair of policies $(f_1, f_2) \in \mathcal{F} \times \mathcal{F}$, do as follows:
 1. Choose an attribute $\mathbf{y}_1 \in \mathcal{X}^l$ such that $f_2(\mathbf{y}_1) = 0$.
 2. Compute $\text{ct}_{\mathbf{y}_1} \leftarrow \text{FHABE.Enc}(\text{pp}, \mathbf{y}_1, \text{sk}_{f_1})$.
 3. Output a re-encryption key $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \text{FHABE.Sanitize}(\text{pp}, \text{pk}_{f_2}, \text{ct}_{\mathbf{y}_1})$
- ReEnc($\text{pp}, \text{rk}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct}$) : On input the public parameter pp , a re-encryption key $\text{rk}_{f_1 \rightarrow f_2}$, a public key pk_{f_2} , and a ciphertext ct , do as follows:
 1. If ct is a freshly generated ciphertext under an attribute $\mathbf{x}$, then output $\perp$ if $f_1(\mathbf{x}) \neq 0$.
 2. Choose an attribute $\mathbf{y}_2 \in \mathcal{X}^l$ such that $f_2(\mathbf{y}_2) = 0$.
 3. Compute $\text{ct}_{\mathbf{y}_2} \leftarrow \text{FHABE.Enc}(\text{pp}, \mathbf{y}_2, \text{ct})$.
 4. Compute $\bar{\text{ct}}_{f_2} \leftarrow \text{FHABE.Eval}(\text{pp}, \text{pk}_{f_2}, (\text{rk}_{f_1 \rightarrow f_2}, \text{ct}_{\mathbf{y}_2}), \text{FHABE.Dec})$.
 5. Output a re-encrypted ciphertext $\hat{\text{ct}}_{f_2} \leftarrow \text{FHABE.Sanitize}(\text{pp}, \text{pk}_{f_2}, \bar{\text{ct}}_{f_2})$.

The following theorem shows the correctness of ABPRE:

Theorem 1. *The ABPRE scheme ABPRE is correct if the FHABE scheme FHABE is sanitizable and correct.*

Proof. First, since Setup, KeyGen, Enc, Dec of ABPRE are identical to those of FHABE, the correctness for freshly generated ciphertexts immediately follows from the correctness of FHABE.

Next, we show the correctness for re-encrypted ciphertexts. For any $(f_1, f_2) \in \mathcal{F} \times \mathcal{F}$, let $(\text{pp}, \text{msk}) \leftarrow \text{FHABE.Setup}(1^\lambda, l)$, $(\text{pk}_{f_1}, \text{sk}_{f_1}) \leftarrow \text{FHABE.KeyGen}(\text{pp}, \text{msk}, f_1)$, and $(\text{pk}_{f_2}, \text{sk}_{f_2}) \leftarrow \text{FHABE.KeyGen}(\text{pp}, \text{msk}, f_2)$. A re-encryption key $\text{rk}_{f_1 \rightarrow f_2}$ is computed as $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \text{FHABE.Sanitize}(\text{pp}, \text{pk}_{f_2}, \text{FHABE.Enc}(\text{pp}, \mathbf{y}_1, \text{sk}_{f_1}))$, where $\mathbf{y}_1 \in \mathcal{X}^l$ such that $f_2(\mathbf{y}_1) = 0$.

Let ct be a ciphertext of message $\mu \in \mathcal{M}$ for policy f_1 . For a re-encrypted ciphertext $\hat{\text{ct}}_{f_2} \leftarrow \text{ReEnc}(\text{pp}, \text{rk}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct})$, we have $\hat{\text{ct}}_{f_2} = \text{FHABE.Sanitize}(\text{pp}, \text{pk}_{f_2}, \bar{\text{ct}}_{f_2})$, where $\mathbf{y}_2 \in \mathcal{X}^l$ such that $f_2(\mathbf{y}_2) = 0$, $\text{ct}_{\mathbf{y}_2} \leftarrow \text{FHABE.Enc}(\text{pp}, \mathbf{y}_2, \text{ct})$, and $\bar{\text{ct}}_{f_2} \leftarrow \text{FHABE.Eval}(\text{pp}, \text{pk}_{f_2}, (\text{rk}_{f_1 \rightarrow f_2}, \text{ct}_{\mathbf{y}_2}), \text{FHABE.Dec})$. Since $\text{rk}_{f_1 \rightarrow f_2}$ and $\text{ct}_{\mathbf{y}_2}$ are valid encryptions of sk_{f_1} and ct for policy f_2 , respectively, $\bar{\text{ct}}_{f_2}$ is a valid ciphertext of $\text{Dec}(\text{pp}, \text{sk}_{f_1}, \text{ct}) = \mu$ for policy f_2 . Due to the sanitizability of FHABE, $\hat{\text{ct}}_{f_2}$ is also a valid encryption of μ for policy f_2 . Hence, it holds that $\Pr[\text{Dec}(\text{pp}, \text{sk}_{f_2}, \hat{\text{ct}}_{f_2}) = \mu] \geq 1 - \text{negl}(\lambda)$, and we complete the proof. $\square$

We next show HRA security of our scheme.

Theorem 2. *The ABPRE scheme ABPRE is selectively HRA-secure, assuming that the FHABE scheme FHABE is selectively CPA-secure, sanitizable, and user-key circular-secure.*

Proof. Due to the following Lemmas 3 and 4, ABPRE is both selectively CPA-secure and re-encryption simulatable. Hence, the theorem follows from Lemma 2. $\square$

Lemma 3. *The ABPRE scheme ABPRE is selectively CPA-secure, assuming that the FHABE scheme FHABE is selectively CPA-secure, sanitizable, and user-key circular-secure.*

Proof. In the proof, we consider the sequence of games below. For $i \in \{0, 1, 2\}$, let Win_i be the event of $\beta = \beta'$ in Game i .

Game 0 : This is the original CPA security game defined in Definition 8. Obviously, we have $\text{Adv}_{\text{ABPRE}, \mathcal{A}}^{\text{CPA}}(\lambda) = |\Pr[\text{Win}_0] - 1/2|$.

Game 1 : This is identical to Game 0 except for the following procedure of $\mathcal{O}^{\text{ReKeyGen}}(f_1, f_2)$: If $f_1(\mathbf{x}^*) = 0$ and $f_2(\mathbf{x}^*) = 0$, we change how to generate a re-encryption key $\text{rk}_{f_1 \rightarrow f_2}$, as follows:

1. Compute $\text{ct}_{\mathbf{x}^*} \leftarrow \text{FHABE.Enc}(\text{pp}, \mathbf{x}^*, \text{sk}_{f_1})$, where $\mathbf{x}^* \in \mathcal{X}^l$ is the attribute chosen by $\mathcal{A}$ at the beginning of the game.
2. Output a re-encryption key $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \text{FHABE.Sanitize}(\text{pp}, \text{pk}_{f_2}, \text{ct}_{\mathbf{x}^*})$.

Note that, in Game 0, $\mathcal{O}^{\text{ReKeyGen}}(f_1, f_2)$ chooses $\mathbf{y}_1 \in \mathcal{X}^l$ such that $f_2(\mathbf{y}_1) = 0$ and outputs $\text{rk}_{f_1 \rightarrow f_2}$ obtained by sanitizing $\text{ct}_{\mathbf{y}_1} \leftarrow \text{FHABE.Enc}(\text{pp}, \mathbf{y}_1, \text{sk}_{f_1})$. Since both $\text{ct}_{\mathbf{y}_1}$ and $\text{ct}_{\mathbf{x}^*}$ are encryptions of the same message sk_{f_1} for policy f_2 , $\text{FHABE.Sanitize}(\text{pp}, \text{pk}_{f_2}, \text{ct}_{\mathbf{y}_1})$ and $\text{FHABE.Sanitize}(\text{pp}, \text{pk}_{f_2}, \text{ct}_{\mathbf{x}^*})$ are statistically close due to the sanitizability of FHABE. Hence, we have $|\Pr[\text{Win}_0] - \Pr[\text{Win}_1]| \leq \text{negl}(\lambda)$.

Game 2 : This is identical to Game 1 except for the following procedure of $\mathcal{O}^{\text{ReKeyGen}}(f_1, f_2)$: If $f_1(\mathbf{x}^*) = 0$ and $f_2(\mathbf{x}^*) = 0$, we change how to generate a re-encryption key $\text{rk}_{f_1 \rightarrow f_2}$, as follows:

1. Sample $\mathbf{r} \xleftarrow{\$} \mathcal{M}$ at random such that $|\mathbf{r}| = |\text{sk}_{f_1}|$.
2. Compute $\text{ct}_{\mathbf{x}^*} \leftarrow \text{FHABE.Enc}(\text{pp}, \mathbf{x}^*, \mathbf{r})$.
3. Output a re-encryption key $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \text{FHABE.Sanitize}(\text{pp}, \text{pk}_{f_2}, \text{ct}_{\mathbf{x}^*})$.

Note that the challenger in Game 2 no longer uses the secret key sk_f such that $f(\mathbf{x}^*) = 0$.

The only difference between Game 1 and Game 2 lies in the way $\text{ct}_{\mathbf{x}^*}$ is computed: it is an encryption of the secret key sk_{f_1} in Game 1, and an encryption of random vector $\mathbf{r}$ in Game 2. To show the indistinguishability between Game 1 and Game 2, we construct an adversary $\mathcal{B}_1$ against user-key circular security of FHABE as follows:

- Given the target attribute $\mathbf{x}^* \in \mathcal{X}^l$ from $\mathcal{A}$, $\mathcal{B}_1$ sends $\mathbf{x}^*$ to the challenger of $\text{Exp}_{\text{FHABE}, \mathcal{B}_1}^{\text{uCircular}}(\lambda)$. The challenger runs $(\text{pp}, \text{msk}) \leftarrow \text{FHABE.Setup}(1^\lambda, l)$, gives pp to $\mathcal{B}_1$ and chooses $\beta \xleftarrow{\$} \{0, 1\}$. Then, $\mathcal{B}_1$ returns pp and initializes an empty key-value store $\mathcal{RK}$.
- Given a key generation query $f \in \mathcal{F}$ from $\mathcal{A}$, $\mathcal{B}_1$ returns pk_f to $\mathcal{A}$ if $f(\mathbf{x}^*) = 0$ and returns $(\text{pk}_f, \text{sk}_f)$ otherwise, where $(\text{pk}_f, \text{sk}_f) \leftarrow \text{FHABE.KeyGen}(\text{pp}, \text{msk}, f)$. This is possible because $\mathcal{B}_1$ can query $\mathcal{O}^{\text{KeyGen}}(f)$ in $\text{Exp}_{\text{FHABE}, \mathcal{B}_1}^{\text{uCircular}}(\lambda)$.

- Given a re-encryption key generation query $(f_1, f_2) \in \mathcal{F} \times \mathcal{F}$, if there exists $\mathcal{RK}[f_1, f_2]$, $\mathcal{B}_1$ retrieve $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \mathcal{RK}[f_1, f_2]$ and returns $\text{rk}_{f_1 \rightarrow f_2}$ to $\mathcal{A}$. Otherwise, $\mathcal{B}_1$ proceeds as follows:
 - If $f_1(\mathbf{x}^*) = 0$ and $f_2(\mathbf{x}^*) \neq 0$, $\mathcal{B}_1$ returns $\perp$ to $\mathcal{A}$.
 - If $f_1(\mathbf{x}^*) = 0$ and $f_2(\mathbf{x}^*) = 0$, $\mathcal{B}_1$ queries $\mathcal{O}^{\text{Enc}}(f_1), \mathcal{O}^{\text{KeyGen}}(f_2)$ and obtains $\text{ct}_{\mathbf{x}^*}, \text{pk}_{f_2}$. Then, $\mathcal{B}_1$ computes $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \text{FHABE.Sanitize}(\text{pp}, \text{pk}_{f_2}, \text{ct}_{\mathbf{x}^*})$, returns $\text{rk}_{f_1 \rightarrow f_2}$ to $\mathcal{A}$, and sets $\mathcal{RK}[f_1, f_2] \leftarrow \text{rk}_{f_1 \rightarrow f_2}$.
 - Otherwise, $\mathcal{B}_1$ queries $\mathcal{O}^{\text{KeyGen}}(f_1), \mathcal{O}^{\text{KeyGen}}(f_2)$ and obtains $\text{sk}_{f_1}, \text{pk}_{f_2}$. Then, $\mathcal{B}_1$ computes $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \text{ReKeyGen}(\text{pp}, \text{sk}_{f_1}, \text{pk}_{f_2}, f_1, f_2)$, returns $\text{rk}_{f_1 \rightarrow f_2}$ to $\mathcal{A}$, and sets $\mathcal{RK}[f_1, f_2] \leftarrow \text{rk}_{f_1 \rightarrow f_2}$.
- When $\mathcal{A}$ submits a pair of messages (μ_0, μ_1) , $\mathcal{B}_1$ chooses $b \xleftarrow{\$} \{0, 1\}$ and returns $\text{ct}^* \leftarrow \text{FHABE.Enc}(\text{pp}, \mathbf{x}^*, \mu_b)$.
- At the end of the game, $\mathcal{B}_1$ receives $b' \in \{0, 1\}$ from $\mathcal{A}$. $\mathcal{B}_1$ outputs $\beta' = 1$ if $b' = b$ and $\beta' = 0$ if $b' \neq b$.

We observe that $\mathcal{B}_1$ perfectly simulates Game 1 if $\beta = 1$ and Game 2 if $\beta = 0$, and thus we have the advantage

$$\begin{aligned}
 \text{Adv}_{\text{FHABE}, \mathcal{B}_1}^{\text{uCircular}}(\lambda) &= |\Pr[\beta' = \beta] - 1/2| \\
 &= |\Pr[\beta = 0] \Pr[b' = b | \beta = 1] + \Pr[\beta = 1] \Pr[b' \neq b | \beta = 0] - 1/2| \\
 &= |1/2 \cdot \Pr[\text{Win}_1] + 1/2 \cdot (1 - \Pr[\text{Win}_2]) - 1/2| \\
 &= 1/2 \cdot |\Pr[\text{Win}_1] - \Pr[\text{Win}_2]|.
 \end{aligned}$$

By the user-key circular security of FHABE, it holds that $|\Pr[\text{Win}_1] - \Pr[\text{Win}_2]| = 2 \cdot \text{Adv}_{\text{FHABE}, \mathcal{B}_1}^{\text{uCircular}}(\lambda) \leq \text{negl}(\lambda)$.

Next, to evaluate $|\Pr[\text{Win}_2] - 1/2|$, we construct a CPA adversary $\mathcal{B}_2$ for FHABE as follows:

- Given the target attribute $\mathbf{x}^* \in \mathcal{X}^l$ from $\mathcal{A}$, $\mathcal{B}_2$ sends $\mathbf{x}^*$ to the CPA challenger for FHABE. The challenger runs $(\text{pp}, \text{msk}) \leftarrow \text{FHABE.Setup}(1^\lambda, l)$ and gives pp to $\mathcal{B}_2$. Then, $\mathcal{B}_2$ returns pp to $\mathcal{A}$ and initializes an empty key-value store $\mathcal{RK}$.
- In **Query Phase 1** and **Query Phase 2**, $\mathcal{B}_2$ answers all valid queries from $\mathcal{A}$ as in Game 2. This is possible because $\mathcal{B}_2$ can obtain all secret keys sk_f such that $f(\mathbf{x}^*) \neq 0$ by querying the oracle $\mathcal{O}^{\text{KeyGen}}(f)$ in $\text{Exp}_{\text{FHABE}, \mathcal{B}_2}^{\text{CPA}}(\lambda)$.
- When $\mathcal{A}$ submits a pair of messages (μ_0, μ_1) , $\mathcal{B}_2$ also submits the same pair to the challenger and receives the challenge ciphertext $\text{ct}^* \leftarrow \text{FHABE.Enc}(\text{pp}, \mathbf{x}^*, \mu_\beta)$, where $\beta \xleftarrow{\$} \{0, 1\}$. Then, $\mathcal{B}_2$ returns ct^* to $\mathcal{A}$.
- At the end of the game, $\mathcal{B}_2$ receives a bit $\beta' \in \{0, 1\}$ from $\mathcal{A}$, and outputs β' .

$\mathcal{B}_2$ simulates Game 1 perfectly, and if $\mathcal{A}$ wins, then $\mathcal{B}_2$ also wins. Hence, by CPA security of FHABE, it holds that $|\Pr[\text{Win}_1] - 1/2| = \text{Adv}_{\text{FHABE}, \mathcal{B}_2}^{\text{CPA}}(\lambda) \leq \text{negl}(\lambda)$.

Finally, we have the advantage

$$\begin{aligned} \text{Adv}_{\text{ABPRE}, \mathcal{A}}^{\text{CPA}}(\lambda) &\leq |\Pr[\text{Win}_0] - \Pr[\text{Win}_1]| + |\Pr[\text{Win}_1] - \Pr[\text{Win}_2]| + |\Pr[\text{Win}_2] - 1/2| \\ &= 2 \cdot \text{Adv}_{\text{FHABE}, \mathcal{B}_1}^{\text{mCircular}}(\lambda) + \text{Adv}_{\text{FHABE}, \mathcal{B}_2}^{\text{CPA}}(\lambda) + \text{negl}(\lambda) \end{aligned}$$

and hence we complete the proof. $\square$

Lemma 4. *The ABPRE scheme ABPRE satisfies re-encryption simulatability if the FHABE scheme FHABE is sanitizable.*

Proof. For every $(f_1, f_2) \in \mathcal{F} \times \mathcal{F}$ and $\mu \in \mathcal{M}$, let $(\text{pp}, \text{msk}) \leftarrow \text{Setup}(1^\lambda, l)$, $(\text{pk}_{f_1}, \text{sk}_{f_1}) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f_1)$, $(\text{pk}_{f_2}, \text{sk}_{f_2}) \leftarrow \text{KeyGen}(\text{pp}, \text{msk}, f_2)$, $\text{rk}_{f_1 \rightarrow f_2} \leftarrow \text{ReKeyGen}(\text{pp}, \text{sk}_{f_1}, \text{pk}_{f_2}, f_1, f_2)$. Let ct be a ciphertext of message μ for policy f_1 , and set $\text{aux} = (\text{pp}, f_1, f_2, \text{pk}_{f_1}, \text{pk}_{f_2}, \text{sk}_{f_2}, \text{ct}, \mu)$. Then, we construct a PPT algorithm ReEncSim as follows:

$$\text{ReEncSim}(\text{aux}) = \text{FHABE.Sanitize}(\text{pp}, \text{pk}_{f_2}, \text{FHABE.Enc}(\text{pp}, \mathbf{x}_2, \mu)),$$

where $\mathbf{x}_2 \in \mathcal{X}^l$ satisfies $f_2(\mathbf{x}_2) = 0$.

Since both $\text{ReEncSim}(\text{aux})$ and $\text{ReEnc}(\text{pp}, \text{rk}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct})$ are sanitized ciphertexts of message μ for policy f_2 , by the sanitizability of FHABE, it holds that

$$\begin{aligned} &\Delta((\text{ReEncSim}(\text{aux}), \text{aux}), (\text{ReEnc}(\text{pp}, \text{rk}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct}), \text{aux})) \\ &\leq \Delta((\text{ReEncSim}(\text{aux}), \text{pp}, \text{pk}_{f_2}, \text{sk}_{f_2}), (\text{ReEnc}(\text{pp}, \text{rk}_{f_1 \rightarrow f_2}, \text{pk}_{f_2}, \text{ct}), \text{pp}, \text{pk}_{f_2}, \text{sk}_{f_2})) \\ &\leq 2^{-\lambda} = \text{negl}(\lambda), \end{aligned}$$

and we complete the proof. $\square$

Acknowledgments. This work was supported by JST K Program Grant Number JPMJKP24U2, Japan. This work was in part supported by JSPS KAKENHI Grant Number JP26K02910.

References

1. Ateniese, G., Benson, K., Hohenberger, S.: Key-private proxy re-encryption. In: CT-RSA. LNCS, vol. 5473, pp. 279–294. Springer (2009)
2. Ateniese, G., Fu, K., Green, M., Hohenberger, S.: Improved proxy re-encryption schemes with applications to secure distributed storage. In: NDSS. The Internet Society (2005)
3. Ateniese, G., Fu, K., Green, M., Hohenberger, S.: Improved proxy re-encryption schemes with applications to secure distributed storage. ACM Trans. Inf. Syst. Secur. **9**(1), 1–30 (2006)
4. Blaze, M., Bleumer, G., Strauss, M.: Divertible protocols and atomic proxy cryptography. In: International conference on the theory and applications of cryptographic techniques. pp. 127–144. Springer (1998)

5. Canard, S., Devigne, J., Laguillaumie, F.: Improving the security of an efficient unidirectional proxy re-encryption scheme. *J. Internet Serv. Inf. Secur.* **1**(2/3), 140–160 (2011)
6. Canetti, R., Hohenberger, S.: Chosen-ciphertext secure proxy re-encryption. In: *CCS*. pp. 185–194. ACM (2007)
7. Chandran, N., Chase, M., Liu, F., Nishimaki, R., Xagawa, K.: Re-encryption, functional re-encryption, and multi-hop re-encryption: A framework for achieving obfuscation-based security and instantiations from lattices. In: *Public Key Cryptography. LNCS*, vol. 8383, pp. 95–112. Springer (2014)
8. Chow, S.S.M., Weng, J., Yang, Y., Deng, R.H.: Efficient unidirectional proxy re-encryption. In: *AFRICACRYPT. LNCS*, vol. 6055, pp. 316–332. Springer (2010)
9. Cohen, A.: What about bob? the inadequacy of CPA security for proxy re-encryption. In: *IACR International Workshop on Public Key Cryptography*. pp. 287–316. Springer (2019)
10. Ducas, L., Stehlé, D.: Sanitization of FHE ciphertexts. In: *Annual International Conference on the Theory and Applications of Cryptographic Techniques*. pp. 294–310. Springer (2016)
11. Fan, X., Liu, F.: Proxy re-encryption and re-signatures from lattices. In: *ACNS. LNCS*, vol. 11464, pp. 363–382. Springer (2019)
12. Fuchsbaauer, G., Kamath, C., Klein, K., Pietrzak, K.: Adaptively secure proxy re-encryption. In: *IACR International Workshop on Public Key Cryptography*. pp. 317–346. Springer (2019)
13. Gentry, C.: A fully homomorphic encryption scheme. Ph.D. thesis, Stanford University, USA (2009), <https://searchworks.stanford.edu/view/8493082>
14. Liang, X., Cao, Z., Lin, H., Shao, J.: Attribute based proxy re-encryption with delegating capabilities. In: *Proceedings of the 4th international symposium on information, computer, and communications security*. pp. 276–286 (2009)
15. Libert, B., Vergnaud, D.: Unidirectional chosen-ciphertext secure proxy re-encryption. In: *Public Key Cryptography. LNCS*, vol. 4939, pp. 360–379. Springer (2008)
16. Sato, S., Shikata, J.: Bounded cca2-secure proxy re-encryption from lattices. In: *SAC*. pp. 425–455. LNCS, Springer (2025)
17. Susilo, W., Dutta, P., Duong, D.H., Roy, P.S.: Lattice-based HRA-secure attribute-based proxy re-encryption in standard model. In: *European Symposium on Research in Computer Security*. pp. 169–191. Springer (2021)
18. Wan, X., Wang, Y., Xue, H., Wang, M.: Unbounded multi-hop proxy re-encryption with HRA security: An lwe-based optimization. In: *ACISP (2). LNCS*, vol. 15659, pp. 124–144. Springer (2025)
19. Yin, H., Zhang, J., Li, W., Dong, Y., Lim, E.G., Wojtczak, D.: Reducing honest re-encryption attack to chosen ciphertext attack. In: *Cryptology ePrint Archive* (2025)
20. Zhao, F., Chen, S., Au, M.H., Weng, J., Wang, H., Guo, J.: Bootstrappable fully homomorphic attribute-based encryption with unbounded circuit depth. In: *International Conference on the Theory and Application of Cryptology and Information Security*. pp. 230–262. Springer (2025)
21. Zhao, F., Weng, J., Xie, W., Hou, L., Li, M.: Time-based attribute-based proxy re-encryption with decryption key update. *Designs, Codes and Cryptography* **92**(12), 4099–4129 (2024)
22. Zhao, F., Weng, J., Xie, W., Li, M., Weng, J.: HRA-secure attribute-based threshold proxy re-encryption from lattices. *Information Sciences* **655**, 119900 (2024)

23. Zhou, Y., Liu, S., Han, S.: Multi-hop fine-grained proxy re-encryption. In: Public Key Cryptography (4). LNCS, vol. 14604, pp. 161–192. Springer (2024)
24. Zhou, Y., Liu, S., Han, S., Zhang, H.: Fine-grained proxy re-encryption: Definitions and constructions from LWE. In: ASIACRYPT (6). LNCS, vol. 14443, pp. 199–231. Springer (2023)

A Construction of Zhao et al.’s FHABE scheme

We describe Zhao et al.’s FHABE scheme [20]. To review this scheme, we describe several notation and trapdoor algorithms related to lattices.

Lattices and its Algorithms. As additional notation, for a matrix $\mathbf{A}$, we define $\bar{\mathbf{A}}$ as the Gram-Schmidt (GS) orthogonalization of $\mathbf{A}$ and $\|\mathbf{A}\|_{\text{GS}} = \|\bar{\mathbf{A}}\|$. For positive integers n, m, q , a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a vector $\mathbf{u} \in \mathbb{Z}_q^n$, we consider the following lattices: $\Lambda_q^\perp(\mathbf{A}) = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}\mathbf{x} = 0 \bmod q\}$ and $\Lambda_q^{\mathbf{u}} = \{\mathbf{x} \in \mathbb{Z}^m \mid \mathbf{A}\mathbf{x} = \mathbf{u} \bmod q\}$.

Zhao et al.’s FHABE employs the following polynomial-time algorithms proposed in existing results:

- $\text{TrapGen}(1^n, 1^m, q) \rightarrow (\mathbf{A}, \mathbf{T}_{\mathbf{A}})$: On input $m = \Theta(n \log q)$, the randomized algorithm TrapGen outputs a full-rank matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a trapdoor basis $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}^{m \times m}$ of $\Lambda_q^\perp(\mathbf{A})$.
- $\text{ExtendRight}(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \mathbf{B}) \rightarrow \mathbf{T}_{[\mathbf{A}|\mathbf{B}]}$: On input full-rank matrices $\mathbf{A}, \mathbf{B} \in \mathbb{Z}_q^{n \times m}$ and a trapdoor $\mathbf{T}_{\mathbf{A}}$ of $\Lambda_q^\perp(\mathbf{A})$, the deterministic algorithm ExtendRight outputs a trapdoor basis $\mathbf{T}_{[\mathbf{A}|\mathbf{B}]}$ of $\Lambda_q^\perp(\mathbf{A}|\mathbf{B})$.
- $\text{SampleD}(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \mathbf{U}, \alpha) \rightarrow \mathbf{R}$: On input a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, a trapdoor $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}^{m \times m}$ of $\Lambda_q^\perp(\mathbf{A})$, a matrix $\mathbf{U} \in \mathbb{Z}_q^{n \times k}$, and a parameter $\alpha = \|\mathbf{T}_{\mathbf{A}}\|_{\text{GS}} \cdot \omega(\sqrt{\log m})$, the randomized algorithm SampleD outputs $\mathbf{R} \in \mathbb{Z}_q^{m \times k}$ which is statistically close to $\mathcal{D}_\alpha(\Lambda_q^{\mathbf{U}}(\mathbf{A}))$ satisfying $\mathbf{A}\mathbf{R} = \mathbf{U}$.

We describe the following key-homomorphic evaluation (deterministic) algorithms employed in [20]:

- $\text{KeyEval}_{\text{pk}}(f, \{\mathbf{B}_1, \dots, \mathbf{B}_l\}) \rightarrow \mathbf{B}_f$: On input a function $f \in \mathcal{F}$ and matrices $\mathbf{B}_1, \dots, \mathbf{B}_l \in \mathbb{Z}_q^{n \times m}$, the deterministic algorithm $\text{KeyEval}_{\text{pk}}$ outputs a matrix $\mathbf{B}_f \in \mathbb{Z}_q^{n \times m}$ associated with f .
- $\text{KeyEval}_{\text{D}}(f, \mathbf{x}, \{\mathbf{B}_1, \dots, \mathbf{B}_l\}) \rightarrow \mathbf{D}_{f, \mathbf{x}}$: On input a policy $f \in \mathcal{F}$, an attribute $\mathbf{x} \in \mathbb{Z}_q^l$, and $\mathbf{B}_1, \dots, \mathbf{B}_l \in \mathbb{Z}_q^{n \times m}$, $\text{KeyEval}_{\text{D}}$ outputs a matrix $\mathbf{D}_{f, \mathbf{x}} \in \mathbb{Z}_q^{lm \times lm}$ such that $[\mathbf{B}_1 + x_1 \mathbf{G}] \cdots [\mathbf{B}_l + x_l \mathbf{G}] \cdot \mathbf{D}_{f, \mathbf{x}} = \mathbf{B}_f + f(\mathbf{x})\mathbf{G}$ and $\|\mathbf{D}_{f, \mathbf{x}}\|_\infty \leq (m+1)^{d_{\mathcal{F}}}$.
- $\text{KeyEval}_{\text{ct}}(f, \mathbf{x}, \{\mathbf{B}_i, \mathbf{c}_i\}_{i \in [l]}) \rightarrow \mathbf{c}_f$: On input $f \in \mathcal{F}$, $\mathbf{x} \in \mathbb{Z}_q^l$, $\mathbf{B}_i \in \mathbb{Z}_q^{n \times m}$, and $\mathbf{c}_i = (\mathbf{B}_i + x_i \mathbf{G})^\top \mathbf{s} + \mathbf{e}_i$, the deterministic algorithm $\text{KeyEval}_{\text{ct}}$ computes $\mathbf{D}_{f, \mathbf{x}} \leftarrow \text{KeyEval}_{\text{D}}(f, \mathbf{x}, \{\mathbf{B}_1, \dots, \mathbf{B}_l\})$ and outputs $\mathbf{c}_f = \mathbf{D}_{f, \mathbf{x}}^\top \cdot [\mathbf{c}_1 \parallel \cdots \parallel \mathbf{c}_l] \in \mathbb{Z}_q^m$ such that $\mathbf{c}_f = [\mathbf{B}_f + f(\mathbf{x})\mathbf{G}]^\top \cdot \mathbf{s} + \mathbf{e}_f$ and $\|\mathbf{e}_f\|_\infty \leq (m+1)^{d_{\mathcal{F}}} \cdot B$.

Zhao et al.'s FHABE construction. The FHABE scheme $\Pi^{\text{FHABE}} = (\text{Setup}, \text{KeyGen}, \text{Enc}, \text{Dec}, \text{Eval})$ employs the following algorithm P2: On input $\mathbf{X} \in \mathbb{Z}_q^{n \times k}$ P2 outputs $\mathbf{G}^\top \cdot \mathbf{X} \in \mathbb{Z}_q^{n \lceil \log q \rceil \times k}$, where $\mathbf{G} = \mathbf{I}_n \otimes (1, 2, \dots, 2^{\lceil \log q \rceil})$.

Then, Π^{FHABE} is constructed as follows:

- $\text{Setup}(1^\lambda) \rightarrow (\text{pp}, \text{msk})$:
 1. Choose parameters, as follows: Let $n = \text{poly}(\lambda)$, $q = 2^d$, $m = \Theta(n \log q)$, $\alpha_{\mathcal{F}} = (m+1)^{d_{\mathcal{F}}}$. Additionally, let $\mathbf{G} \in \mathbb{Z}_q^{n \times m}$ be a gadget matrix and χ be a B_χ -bounded error distribution. Set $\sigma = \omega(\alpha_{\mathcal{F}} \cdot \sqrt{\log m})$, where $\alpha_{\mathcal{F}} > \sqrt{n \log m}$ for ensuring the correctness of the trapdoor algorithms. Let $Q = B_g^{d_g}$ denote the modulus of Ring-LWE, for some base B_g and $N = q/2$ denote its dimension. A bootstrapping base B_r and a key switching base B_{ks} are chosen suitably for the correctness of $\Pi_{\text{ZCA}^+}^{\text{ABPRE}}$.
 2. Generate $(\mathbf{A}, \mathbf{T}_{\mathbf{A}}) \leftarrow \text{TrapGen}(1^n, 1^m, q)$, where $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{T}_{\mathbf{A}} \in \mathbb{Z}_q^{m \times m}$ is a basis of $\Lambda_q^\perp(\mathbf{A})$.
 3. Choose $(l+1)$ random matrices $\mathbf{B}_1, \dots, \mathbf{B}_l, \mathbf{U} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$.
 4. Output $\text{pp} = (n, m, q, \chi, \sigma, N, Q, B_g, B_r, B_{\text{ks}}, \bar{\chi}, \bar{\sigma}, \mathbf{A}, \mathbf{B}_1, \dots, \mathbf{B}_l, \mathbf{U}, \mathbf{G})$ and $\text{msk} = \mathbf{T}_{\mathbf{A}}$.
- $\text{KeyGen}(\text{pp}, \text{msk}, f) \rightarrow (\text{pk}_f, \text{sk}_f)$:
 1. Generate $\mathbf{B}_f \leftarrow \text{KeyEval}_{\text{pk}}(f, \mathbf{B}_1, \dots, \mathbf{B}_l)$.
 2. Compute $\mathbf{T}_{[\mathbf{A}|\mathbf{B}_f]} \leftarrow \text{ExtendRight}(\mathbf{A}, \mathbf{T}_{\mathbf{A}}, \mathbf{B}_f)$.
 3. Sample $\mathbf{R}_f \leftarrow \text{SampleD}(\mathbf{A}|\mathbf{B}_f, \mathbf{T}_{[\mathbf{A}|\mathbf{B}_f]}, \mathbf{U}, \sigma)$ such that $[\mathbf{A}|\mathbf{B}_f]\mathbf{R}_f = \mathbf{U}$.
 4. Let $\mathbf{R}_f = [\mathbf{r}_1 | \dots | \mathbf{r}_f] \in \mathbb{Z}_q^{2m \times m}$ (where $\mathbf{r}_i \in \mathbb{Z}_q^{2m}$ for $i \in [m]$), compute $\text{btk}_f = \{\text{btk}_{\mathbf{r}_k}\}_{k=1}^m$ (where $\text{btk}_{\mathbf{r}_k} = \{\mathbf{Z}_{h,i,j}^{(k)} = \text{RGSW}_z(X^{h \cdot \mathbf{r}_k[i] \cdot B_r^j})\}$ for some secret $z \leftarrow R_Q$, $h \in [0, B_r - 1]$, $i \in [2m]$, $j \in [0, d_r - 1]$, $d_r = \lceil \log_{B_r} q \rceil$).
 5. Compute a key switch key $\text{ksk}_f = \{\text{ksk}_{\mathbf{r}_k}\}_{k=1}^m$ (where $\text{ksk}_{\mathbf{r}_k} = \{\mathbf{K}_{h,i,j}^{(k)} = \text{LWE}_{\mathbf{r}_k}^{q/q}(hz_i^j B_{\text{ks}}^j)\}$ for $\mathbf{z}' = (z_0, -z_{N-1}, \dots, -z_1)$, $h \in [0, B_{\text{ks}} - 1]$, $i \in [N]$, $j \in [0, d_{\text{ks}} - 1]$, and $d_{\text{ks}} = \lceil \log_{B_{\text{ks}}} q \rceil$).
 6. Set $\bar{\mathbf{R}}_f = \begin{bmatrix} \mathbf{r}_1 & \mathbf{0} & \dots & \mathbf{0} \\ \mathbf{0} & \mathbf{r}_2 & \dots & \mathbf{0} \\ \vdots & \vdots & \ddots & \vdots \\ \mathbf{0} & \mathbf{0} & \dots & \mathbf{r}_m \end{bmatrix} \in \mathbb{Z}_q^{2mm \times m}$. Then choose $\mathbf{x}' = (x'_1, \dots, x'_l) \in \mathbb{Z}_q^l$ such that $f(\mathbf{x}') = 0$. Let $\bar{\mathbf{M}}_1 = [\mathbf{A}|\mathbf{B}_1 + x'_1 \mathbf{G} | \dots | \mathbf{B}_l + x'_l \mathbf{G}]^\top \mathbf{E}_1 + \bar{\mathbf{E}}_2$ and $\mathbf{M}_2 = \mathbf{U}^\top \mathbf{E}_1 + \mathbf{E}_3 - \text{P2}(\bar{\mathbf{R}}_f)^\top$, where $\mathbf{E}_1 \xleftarrow{\$} \mathbb{Z}_q^{n \times 2mmd}$, $\bar{\mathbf{E}}_2 \leftarrow \chi^{(l+1)m \times 2mmd}$, and $\mathbf{E}_3 \leftarrow \chi^{m \times 2mmd}$.
 7. Compute $\mathbf{D}_{f,\mathbf{x}'} \leftarrow \text{KeyEval}_{\mathbf{D}}(f, \mathbf{x}', \{\mathbf{B}_1, \dots, \mathbf{B}_l\})$. Then compute auxiliary recovery key $\text{ark}_f = \begin{bmatrix} \mathbf{M}_1 & \mathbf{0}_{2m \times m} \\ \mathbf{M}_2 & \mathbf{I}_{m \times m} \end{bmatrix} \in \mathbb{Z}_q^{3m \times (2mmd+m)}$, where $\mathbf{M}_1 = \begin{bmatrix} \mathbf{I}_{m \times m} & \mathbf{0}_{m \times m} \\ \mathbf{0}_{lm \times m} & \mathbf{D}_{f,\mathbf{x}'} \end{bmatrix}^\top \cdot \bar{\mathbf{M}}_1 = [\mathbf{A}|\mathbf{B}_f]^\top \mathbf{E}_1 + \mathbf{E}_2$, for some $\|\mathbf{E}_2\|_\infty < \alpha_{\mathcal{F}} \cdot B_\chi$.
 8. Output $\text{pk}_f = (\text{btk}_f, \text{ksk}_f, \text{ark}_f)$ and $\text{sk}_f = \mathbf{R}_f$.
- $\text{Enc}(\text{pp}, \mathbf{x}, \boldsymbol{\mu}) \rightarrow \text{ct}_{\mathbf{x}}$:

1. Choose a random vector $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$.
 2. Choose l matrices $\mathbf{S}_i \xleftarrow{\$} \{1, -1\}^{m \times m}$ for $i \in [l]$.
 3. Sample error vectors $\mathbf{e}_0, \mathbf{e}_{out} \leftarrow \chi^m$ and compute $\mathbf{e} = [\mathbf{I}_m | \mathbf{S}_1 | \dots | \mathbf{S}_l]^\top \cdot \mathbf{e}_0 = (\mathbf{e}_{in}, \mathbf{e}_1, \dots, \mathbf{e}_l) \in \mathbb{Z}_q^{(l+1)m}$.
 4. Set $\mathbf{H}_\mathbf{x} = [\mathbf{A} | \mathbf{B}_1 + x_1 \mathbf{G} | \dots | \mathbf{B}_l + x_l \mathbf{G}] \in \mathbb{Z}_q^{n \times (l+1)m}$.
 5. Compute $\mathbf{c} \leftarrow \mathbf{H}_\mathbf{x}^\top \mathbf{s} + \mathbf{e} \in \mathbb{Z}_q^{(l+1)m}$ and $\mathbf{c}_{out} \leftarrow \mathbf{U}^\top \mathbf{s} + \mathbf{e}_{out} + \lfloor q/4 \rfloor \cdot \boldsymbol{\mu} \in \mathbb{Z}_q^m$.
 6. Output $\mathbf{ct}_\mathbf{x} = (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l, \mathbf{c}_{out}) \in \mathbb{Z}_q^{(l+2)m}$.
- Dec(pp, sk_f, ct) → $\boldsymbol{\mu}$:
1. If ct is a fresh ciphertext, let ct be $\mathbf{ct}_\mathbf{x} = (\mathbf{c}_{in}, \mathbf{c}_1, \dots, \mathbf{c}_l, \mathbf{c}_{out})$, output $\perp$ if $f(\mathbf{x}) \neq 0$; otherwise, compute $\text{KeyEval}_{\mathbf{ct}}(f, \mathbf{x}, \{\mathbf{B}_i, \mathbf{c}_i\}_{i=1}^l) \rightarrow \mathbf{c}_f = [\mathbf{B}_f + f(\mathbf{x})\mathbf{G}]^\top \cdot \mathbf{s} + \mathbf{e}_f$, where $\mathbf{e}_f = \mathbf{D}_{f, \mathbf{x}}^\top \cdot [\mathbf{e}_1 || \dots || \mathbf{e}_l]$. Then, let $\hat{\mathbf{c}}_f = (\hat{\mathbf{c}}_f, \mathbf{c}_{out}) \in \mathbb{Z}_q^{3m}$, where $\hat{\mathbf{c}}_f = [\mathbf{c}_{in} || \mathbf{c}_f] = [\mathbf{A} | \mathbf{B}_f + f(\mathbf{x})\mathbf{G}]^\top \mathbf{s} + [\mathbf{e}_{in} || \mathbf{e}_f] \in \mathbb{Z}_q^{2m}$.
 2. If ct is an evaluated ciphertext under f , let ct be $\hat{\mathbf{c}}_f^{(g)} = (\hat{\mathbf{c}}_f, \mathbf{c}_{out}) \in \mathbb{Z}_q^{3m}$.
 3. Output $\boldsymbol{\mu} \leftarrow \left\lfloor (4/q)(\mathbf{c}_{out} - \mathbf{R}_f^\top \cdot \hat{\mathbf{c}}_f) \right\rfloor \bmod 4$, where $\boldsymbol{\mu} \in \{0, 1\}^m$.
- Eval.NAND(pp, pk_f, ct⁽¹⁾, ct⁽²⁾) → $\hat{\mathbf{c}}_f^{(g)}$:
1. For each $i \in \{1, 2\}$ if ct⁽ⁱ⁾ is a fresh ciphertext, let ct⁽ⁱ⁾ be $\mathbf{ct}_{\mathbf{x}^{(i)}} = (\mathbf{c}_{in}^{(i)}, \mathbf{c}_1^{(i)}, \dots, \mathbf{c}_l^{(i)}, \mathbf{c}_{out}^{(i)})$, which satisfies $f(\mathbf{x}^{(i)}) = 0$, compute $\mathbf{c}_f^{(i)} \leftarrow \text{KeyEval}_{\mathbf{ct}}(f, \mathbf{x}^{(i)}, \{\mathbf{B}_j, \mathbf{c}_j^{(i)}\}_{j=1}^l)$, set $\hat{\mathbf{c}}_f^{(i)} = [\hat{\mathbf{c}}_{in}^{(i)} || \mathbf{c}_f^{(i)}] \in \mathbb{Z}_q^{2m}$; if ct⁽ⁱ⁾ is an evaluated ciphertext under f , let ct⁽ⁱ⁾ be $(\hat{\mathbf{c}}_f^{(i)}, \mathbf{c}_{out}^{(i)})$. Then set $\hat{\mathbf{c}}_f^{(i)} = (\hat{\mathbf{c}}_f^{(i)}, \mathbf{c}_{out}^{(i)}) \in \mathbb{Z}_q^{3m}$ for $i \in \{1, 2\}$.
 2. We have $\hat{\mathbf{c}}_f^{(1)} = (\hat{\mathbf{c}}_f^{(1)}, \mathbf{c}_{out}^{(1)})$ and $\hat{\mathbf{c}}_f^{(2)} = (\hat{\mathbf{c}}_f^{(2)}, \mathbf{c}_{out}^{(2)})$. Compute
$$\mathbf{a} = \hat{\mathbf{c}}_f^{(1)} + \hat{\mathbf{c}}_f^{(2)} \in \mathbb{Z}_q^{2m}, \quad \mathbf{b} = (b_1, \dots, b_m) = \hat{\mathbf{c}}_f^{(1)} + \hat{\mathbf{c}}_f^{(2)} \in \mathbb{Z}_q^m.$$
 3. For each $i \in [m]$, do the bootstrapping procedure, as follows:
$$\bar{\mathbf{c}}_i = (\bar{\mathbf{a}}_i, \bar{b}_i) \leftarrow \text{Bootstrap}_{\text{NAND}}(\text{btk}_{r_i}, \text{ksks}_{r_i}, (\mathbf{a}, b)) \in \text{LWE}_{\mathbf{r}_i}^{4/q}(\mu_i^{(1)} \bar{\wedge} \mu_i^{(2)}, \delta)$$
holding $\bar{\mathbf{a}}_i \in \mathbb{Z}_q^{2m}$ and $\bar{b}_i \in \mathbb{Z}_q$ with error bound δ .
 4. Set $\bar{\mathbf{c}}_f = [\bar{\mathbf{a}}_1 || \dots || \bar{\mathbf{a}}_m] \in \mathbb{Z}_q^{2mm}$ and $\bar{\mathbf{c}}_{out} = (\bar{b}_1, \dots, \bar{b}_m) \in \mathbb{Z}_q^m$.
 5. Compute $\hat{\mathbf{c}}_f^{(g)} \leftarrow \text{ark}_f \cdot \begin{bmatrix} \mathbf{G}^{-1}(\bar{\mathbf{c}}_f) \\ \bar{\mathbf{c}}_{out} \end{bmatrix}$
 6. Output $\hat{\mathbf{c}}_f^{(g)} = (\hat{\mathbf{c}}_f^{(g)}, \mathbf{c}_{out}^{(g)}) \in \mathbb{Z}_q^{3m}$.